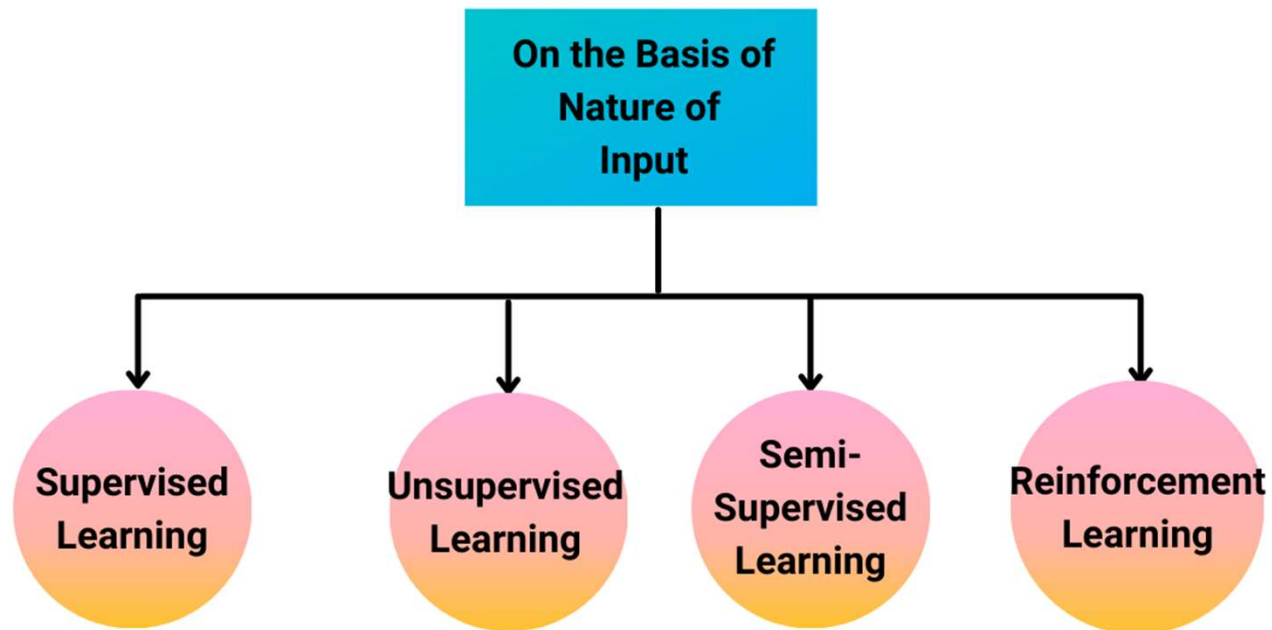


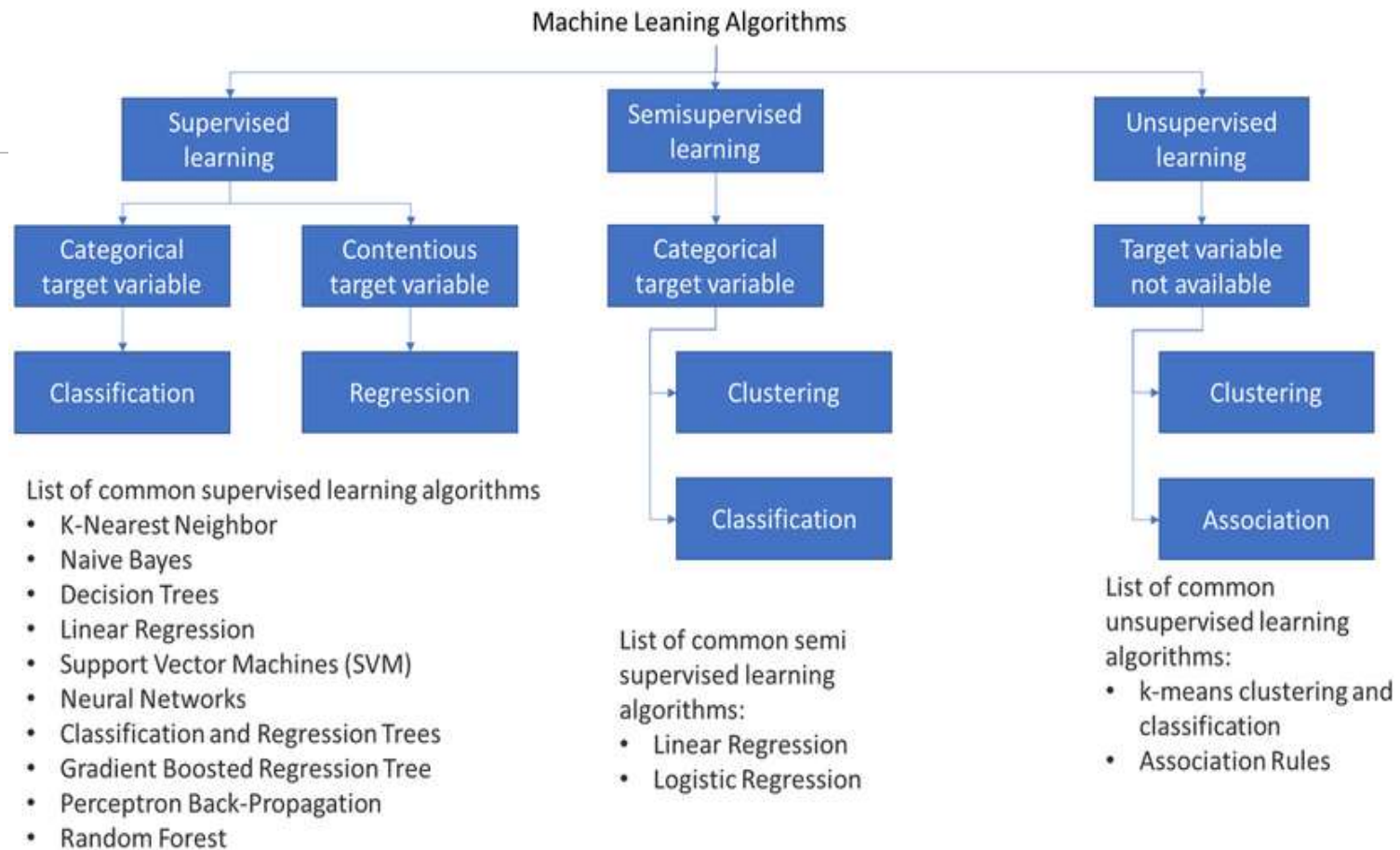
Syllabus

Module No	Module Description	No. of hours	SOs
1	UNIT I Introduction to Machine Learning Preface of Machine Learning, Types of Learning : Supervised - Unsupervised Learning- Reinforcement- theory of learning – feasibility of learning – error and noise – training versus testing – theory of generalization – generalization bound – approximation-generalization tradeoff – bias and variance – learning curve - Finite and Infinite Hypothesis Spaces, Probably Approximately Correct (PAC) Learning-Bayes theorem ,MDL principle.	8	a,b
2	UNIT II Supervised Learning Learning a Class from Examples, Linear, Non-linear, Multi-class and Multi-label classification, Generalization error bounds: VC Dimension, Decision Trees: ID3, Classification and Regression Trees, Regression: Linear Regression, Multiple Linear Regression, Logistic Regression. Neural Networks: Introduction, Perceptron, Multilayer Perceptron, Support vector machines: Linear and Non-Linear, Kernel Functions, K-Nearest Neighbors.	9	a,b,e
3	UNIT III Ensemble Learning and Unsupervised Learning Ensemble Learning Model Combination Schemes, Voting, Error-Correcting Output Codes, Bagging: Random Forest Trees, Boosting: Adaboost. Introduction to clustering: Hierarchical: Partitional: K-means clustering, K- Mode Clustering, Self-Organizing Map, Expectation Maximization, Gaussian Mixture Models- Principal components analysis (PCA), Locally Linear Embedding (LLE), Factor Analysis.	10	a,b,e
4	UNIT V Reinforcement Learning Passive reinforcement learning – Direct utility estimation - Adaptive dynamic programming- Temporal difference learning- Active Reinforcement learning –Exploration –learning an action-utility function -Generalization in reinforcement learning – policy search - Applications.	8	a,b,e
5	Unit V Neural Networks and Machine Learning in Practice Introduction to Neural Networks - Fundamental concepts: neuron models and basic learning rules-Multilayer neural networks and back-propagation - Machine Learning in Practice Design, Analysis and Evaluation of Machine Learning Experiments.	7	a,b,e,i

Machine Learning Classification



Machine Learning Classification



Unsupervised Learning



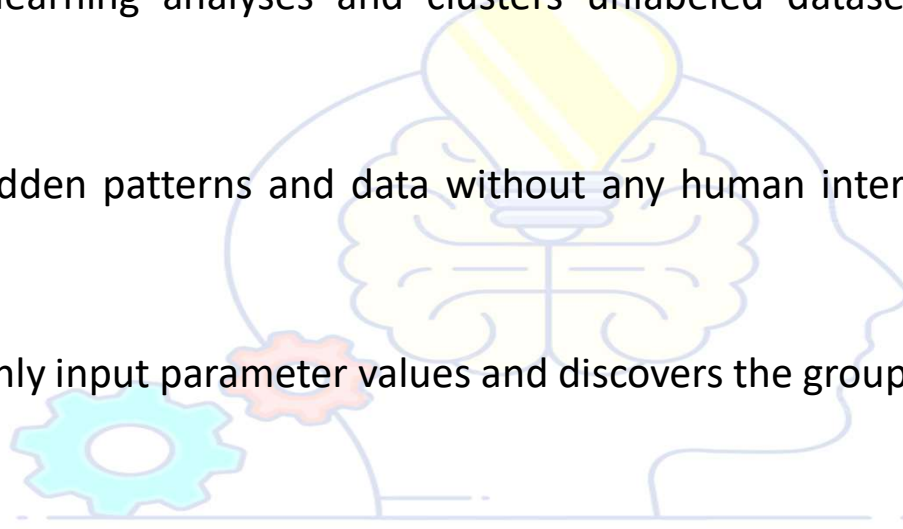
- Unsupervised learning is a branch of machine learning that deals with **unlabeled data**.
- Unlike supervised learning, where the data is labelled with a specific category or outcome, unsupervised learning algorithms are tasked with **finding patterns** and **relationships** within the data without any prior knowledge of the data's meaning.
- This makes unsupervised learning a powerful tool for **exploratory data analysis**, where the goal is to understand the underlying structure of the data.

Machine Learning

Unsupervised Learning



- Unsupervised machine learning analyses and clusters unlabeled datasets using machine learning algorithms.
- These algorithms find hidden patterns and data without any human intervention, i.e., we don't give output to our model.
- The training model has only input parameter values and discovers the groups or patterns on its own.

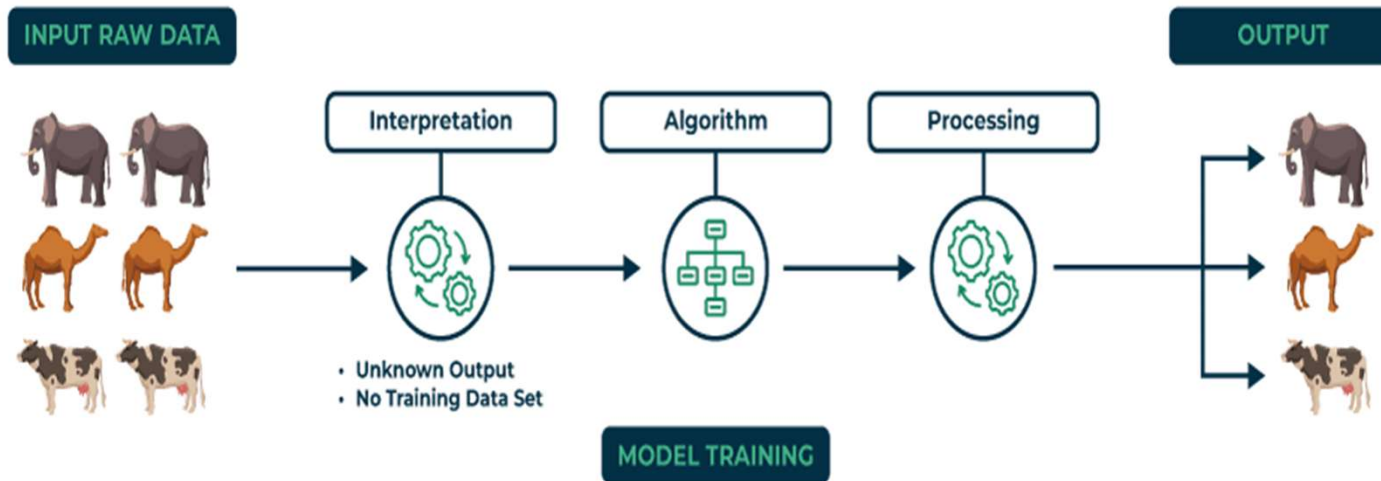


Machine Learning

Unsupervised Learning



Unsupervised Learning

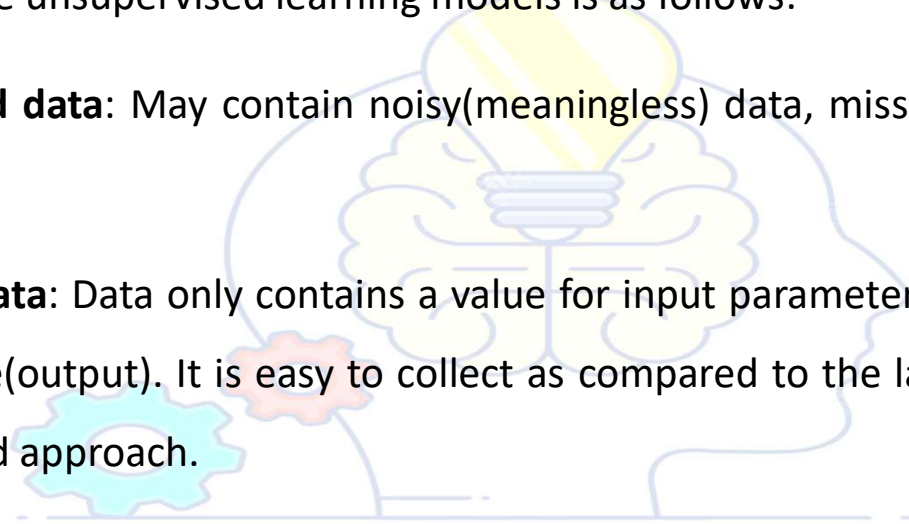


Unsupervised Learning



The input to the unsupervised learning models is as follows:

- ❑ **Unstructured data:** May contain noisy(meaningless) data, missing values, or unknown data
- ❑ **Unlabeled data:** Data only contains a value for input parameters, there is no targeted value(output). It is easy to collect as compared to the labeled one in the Supervised approach.



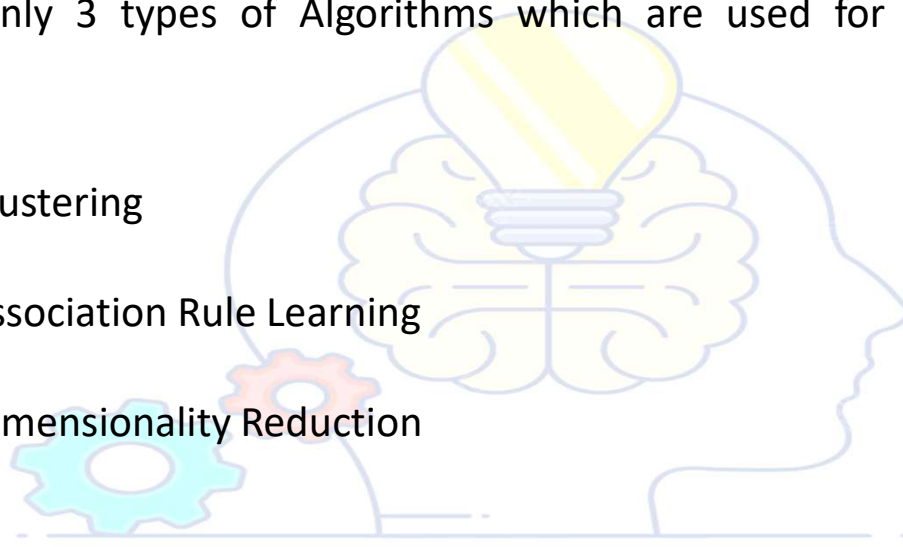
Machine Learning

Unsupervised Learning Algorithm



There are mainly 3 types of Algorithms which are used for Unsupervised dataset.

1. Clustering
2. Association Rule Learning
3. Dimensionality Reduction



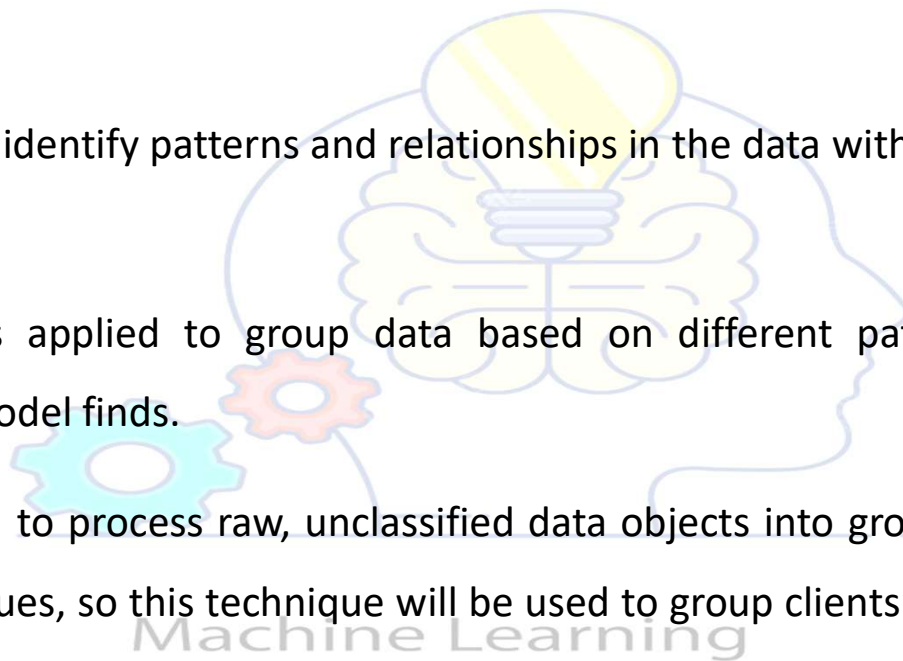
Machine Learning

Unsupervised Learning Algorithm



Clustering

- Clustering in unsupervised machine learning is the process of grouping unlabeled data into clusters based on their similarities.
- The goal of clustering is to identify patterns and relationships in the data without any prior knowledge of the data's meaning.
- Broadly this technique is applied to group data based on different patterns, such as similarities or differences, our machine model finds.
- These algorithms are used to process raw, unclassified data objects into groups. For example, we have not given output parameter values, so this technique will be used to group clients based on the input parameters provided by our data.



Unsupervised Learning Algorithm



Clustering

Some common clustering algorithms

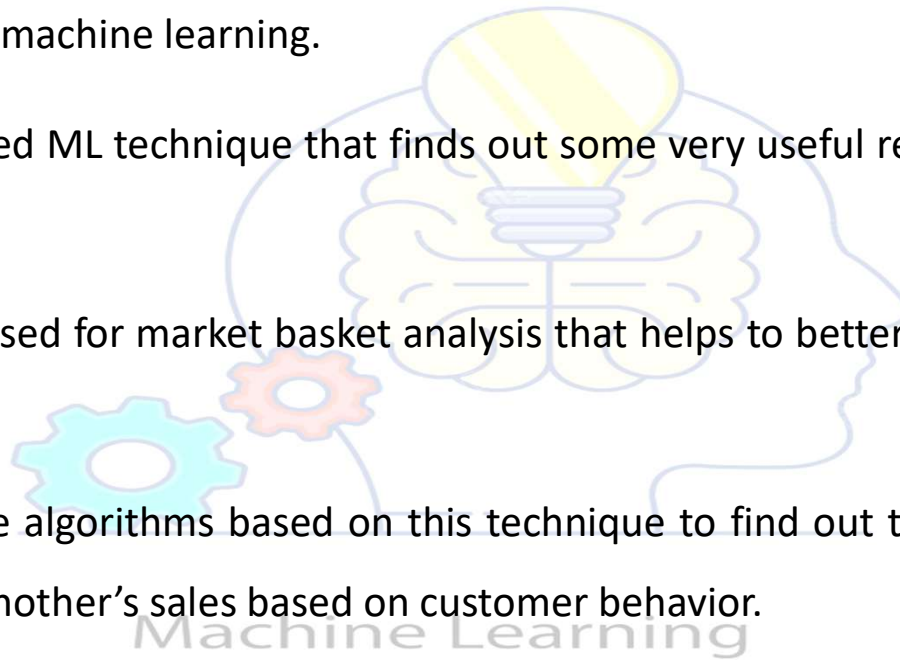
- ❑ **K-means Clustering**: Partitioning Data into K Clusters
- ❑ **Hierarchical Clustering**: Building a Hierarchical Structure of Clusters
- ❑ **Density-Based Clustering (DBSCAN)**: Identifying Clusters Based on Density
- ❑ **Mean-Shift Clustering**: Finding Clusters Based on Mode Seeking
- ❑ **Spectral Clustering**: Utilizing Spectral Graph Theory for Clustering

Unsupervised Learning Algorithm

Association Rule Learning



- Association rule learning is also known as association rule mining is a common technique used to discover associations in unsupervised machine learning.
- This technique is a rule-based ML technique that finds out some very useful relations between parameters of a large data set.
- This technique is basically used for market basket analysis that helps to better understand the relationship between different products.
- For e.g. shopping stores use algorithms based on this technique to find out the relationship between the sale of one product w.r.t to another's sales based on customer behavior.

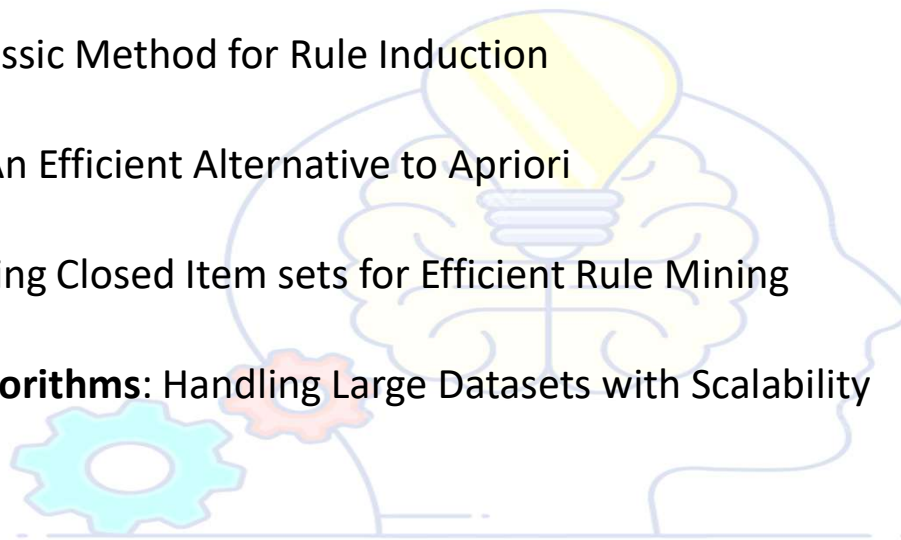


Unsupervised Learning Algorithm



Association Rule Learning

- ❑ **Apriori Algorithm:** A Classic Method for Rule Induction
- ❑ **FP-Growth Algorithm:** An Efficient Alternative to Apriori
- ❑ **Eclat Algorithm:** Exploiting Closed Item sets for Efficient Rule Mining
- ❑ **Efficient Tree-based Algorithms:** Handling Large Datasets with Scalability



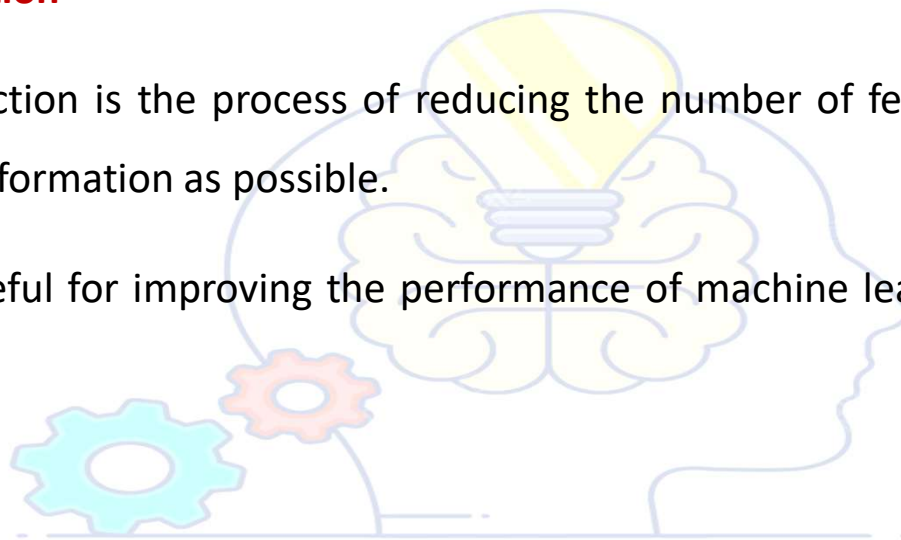
Machine Learning

Unsupervised Learning Algorithm



Dimensionality Reduction

- Dimensionality reduction is the process of reducing the number of features in a dataset while preserving as much information as possible.
- This technique is useful for improving the performance of machine learning algorithms and for data visualization.



Machine Learning

Unsupervised Learning Algorithm



Dimensionality Reduction

Examples of dimensionality reduction algorithms include

- ❑ **Principal Component Analysis (PCA):** Linear Transformation for Reduced Dimensions
- ❑ **Linear Discriminant Analysis (LDA):** Dimensionality Reduction for Discrimination
- ❑ **Non-negative Matrix Factorization (NMF):** Decomposing Data into Non-negative Components
- ❑ **Locally Linear Embedding (LLE):** Preserving Local Geometry in Reduced Dimensions
- ❑ **Isomap:** Capturing Global Relationships in Reduced Dimensions

Machine Learning

Challenges of Unsupervised Learning



- **Evaluation:** Assessing the performance of unsupervised learning algorithms is difficult without predefined labels or categories.
- **Interpretability:** Understanding the decision-making process of unsupervised learning models is often challenging.
- **Overfitting:** Unsupervised learning algorithms can overfit to the specific dataset used for training, limiting their ability to generalize to new data.
- **Data quality:** Unsupervised learning algorithms are sensitive to the quality of the input data. Noisy or incomplete data can lead to misleading or inaccurate results.
- **Computational complexity:** Some unsupervised learning algorithms, particularly those dealing with high-dimensional data or large datasets, can be computationally expensive.

Advantages of Unsupervised learning



- **No labeled data required:** Unlike supervised learning, unsupervised learning does not require labeled data, which can be expensive and time-consuming to collect.
- **Can uncover hidden patterns:** Unsupervised learning algorithms can identify patterns and relationships in data that may not be obvious to humans.
- **Can be used for a variety of tasks:** Unsupervised learning can be used for a variety of tasks, such as clustering, dimensionality reduction, and anomaly detection.
- **Can be used to explore new data:** Unsupervised learning can be used to explore new data and gain insights that may not be possible with other methods.

Disadvantages of Unsupervised learning



- **Difficult to evaluate:** It can be difficult to evaluate the performance of unsupervised learning algorithms, as there are no predefined labels or categories against which to compare results.
- **Can be difficult to interpret:** It can be difficult to understand the decision-making process of unsupervised learning models.
- **Can be sensitive to the quality of the data:** Unsupervised learning algorithms can be sensitive to the quality of the input data. Noisy or incomplete data can lead to misleading or inaccurate results.
- **Can be computationally expensive:** Some unsupervised learning algorithms, particularly those dealing with high-dimensional data or large datasets, can be computationally expensive

Machine Learning

Applications of Unsupervised learning



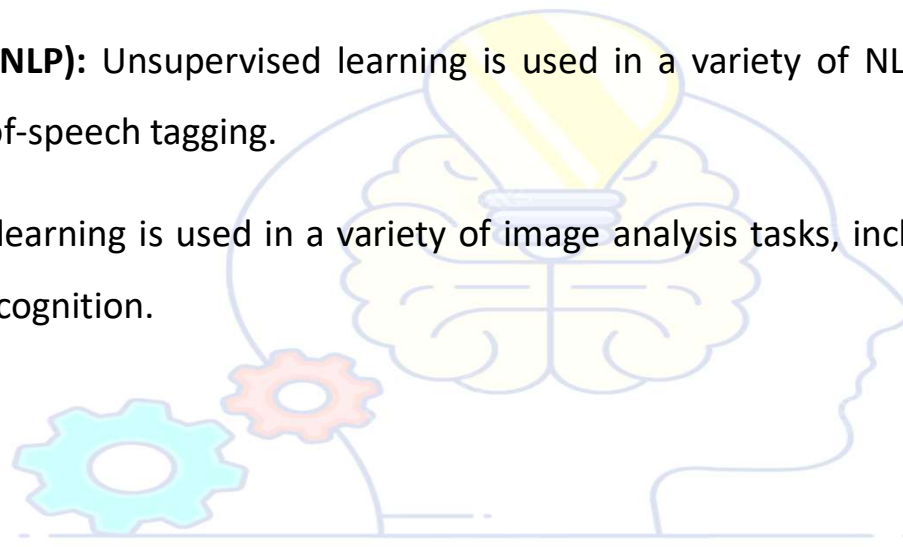
- **Customer segmentation:** can be used to segment customers into groups based on their demographics, behavior, or preferences. This can help businesses to better understand their customers and target them with more relevant marketing campaigns.
- **Fraud detection:** can be used to detect fraud in financial data by identifying transactions that deviate from the expected patterns. This can help to prevent fraud by flagging these transactions for further investigation.
- **Recommendation systems:** can be used to recommend items to users based on their past behavior or preferences. For example, a recommendation system might use unsupervised learning to identify users who have similar taste in movies, and then recommend movies that those users have enjoyed.

Machine Learning

Applications of Unsupervised learning



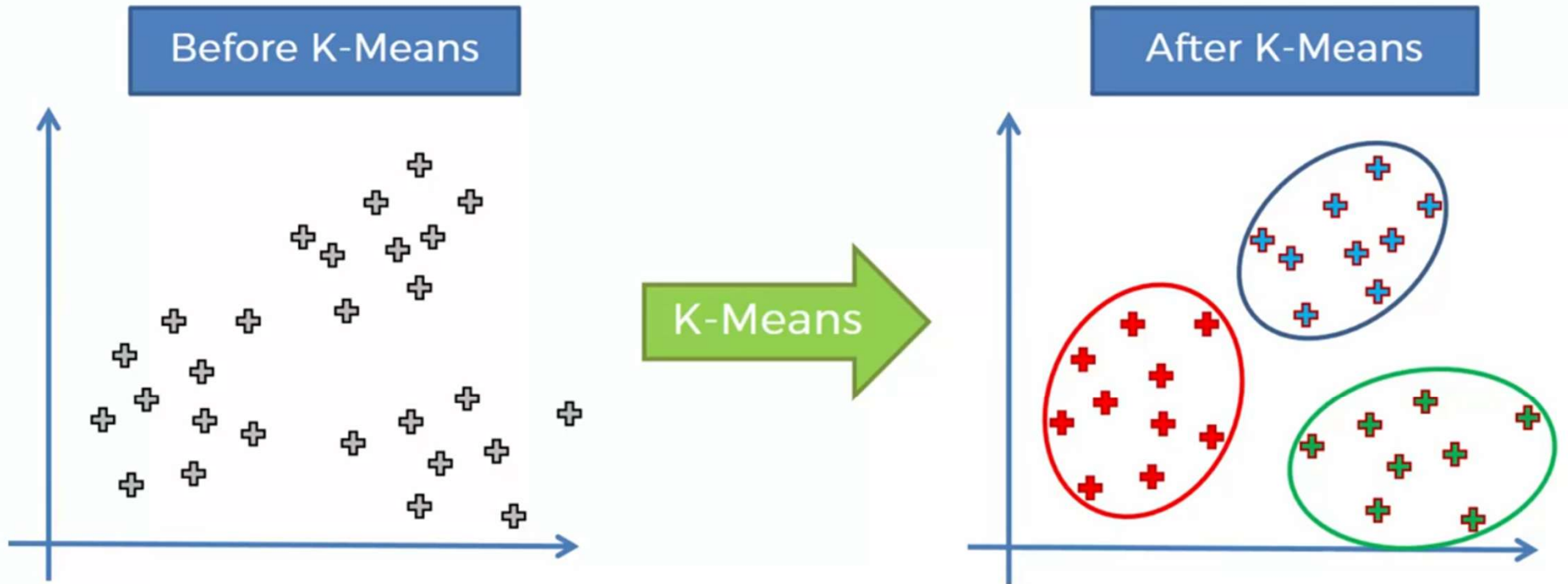
- **Natural language processing (NLP):** Unsupervised learning is used in a variety of NLP tasks, including topic modeling, document clustering, and part-of-speech tagging.
- **Image analysis:** Unsupervised learning is used in a variety of image analysis tasks, including image segmentation, object detection, and image pattern recognition.



Machine Learning

K-Means Intuition: Understanding K-Means

What K-Means does for you



How did it do that ?

STEP 1: Choose the number K of clusters



STEP 2: Select at random K points, the centroids (not necessarily from your dataset)



STEP 3: Assign each data point to the closest centroid → That forms K clusters



STEP 4: Compute and place the new centroid of each cluster



STEP 5: Reassign each data point to the new closest centroid.
If any reassignment took place, go to STEP 4, otherwise go to FIN.



Your Model is Ready

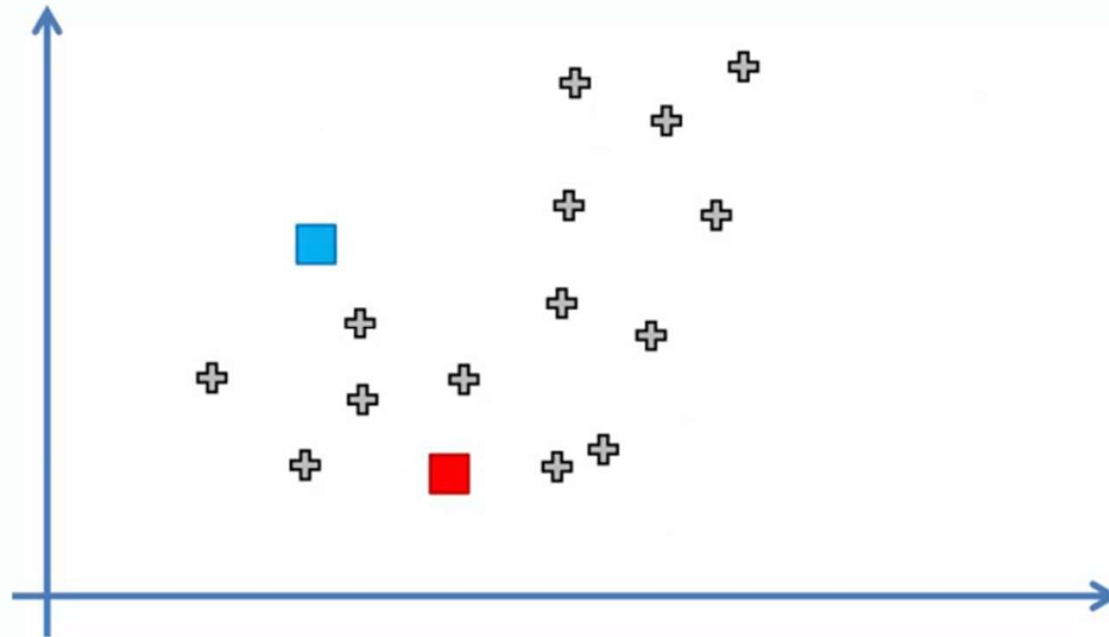
K-Means algorithm

STEP 1: Choose the number K of clusters: $K = 2$



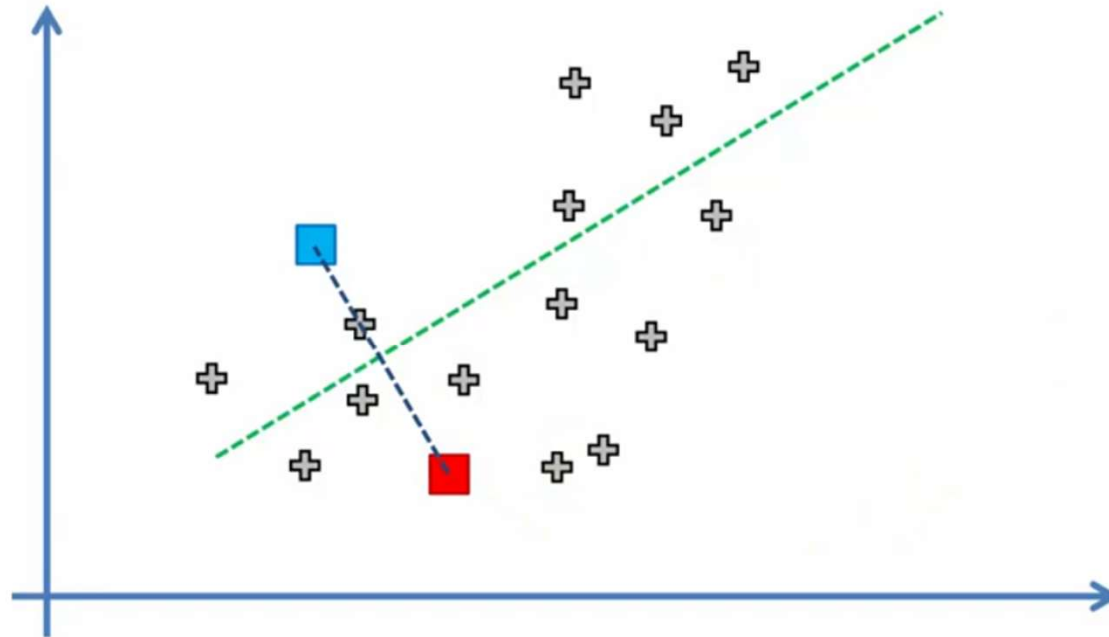
K-Means algorithm

STEP 2: Select at random K points, the centroids (not necessarily from your dataset)



K-Means algorithm

STEP 3: Assign each data point to the closest centroid → That forms K clusters



K-Means Clustering



- K-Means Clustering is an Unsupervised Machine Learning algorithm, which groups the unlabeled dataset into different clusters.
- K means clustering, assigns data points to one of the K clusters depending on their distance from the center of the clusters. It starts by randomly assigning the clusters centroid in the space.
- Then each data point assign to one of the cluster based on its distance from centroid of the cluster. After assigning each point to one of the cluster, new cluster centroids are assigned. This process runs iteratively until it finds good cluster.

Machine Learning

K-Means Clustering

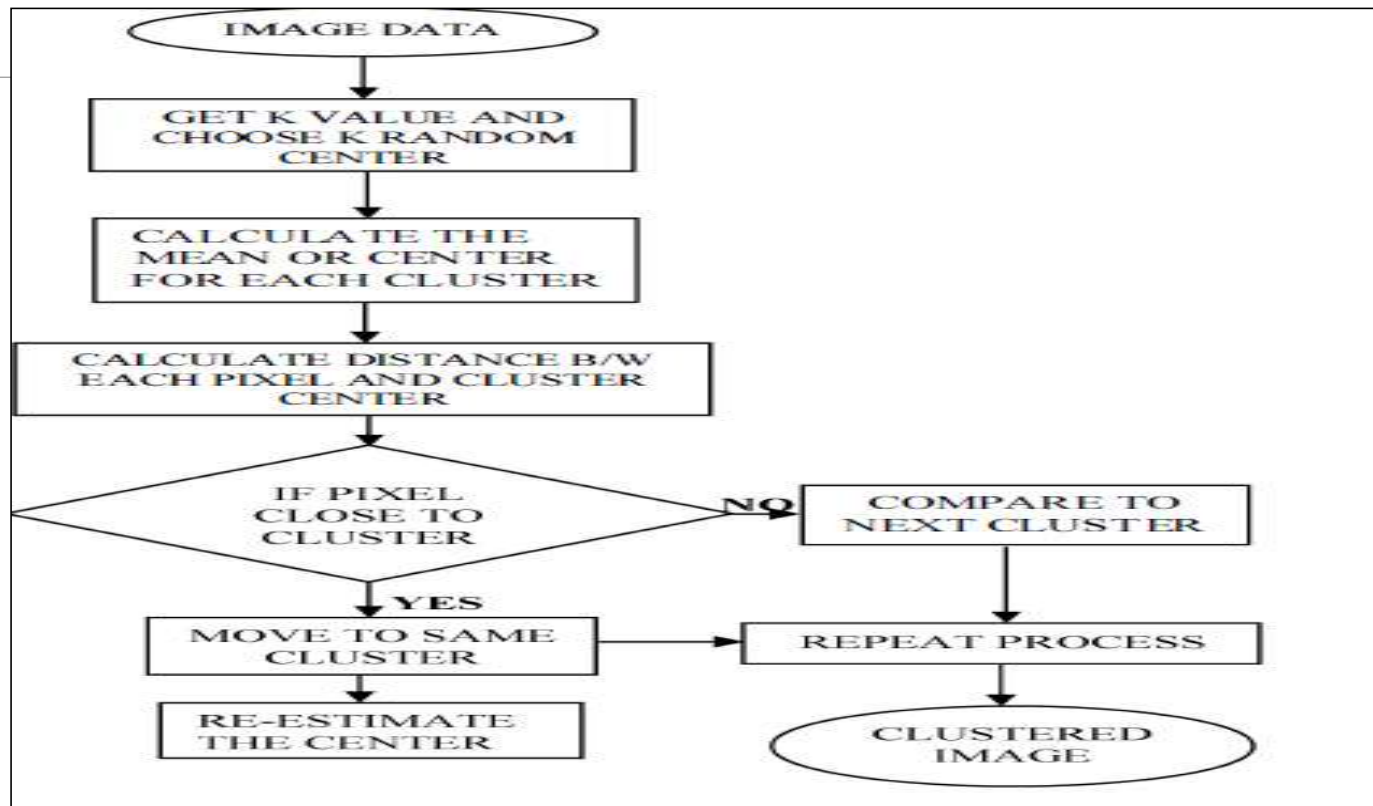


- In some cases, K is not clearly defined, and we have to think about the optimal number of K . K Means clustering performs best data is well separated. When data points overlapped this clustering is not suitable.
- K Means is faster as compare to other clustering technique. It provides strong coupling between the data points. K Means cluster do not provide clear information regarding the quality of clusters.
- Different initial assignment of cluster centroid may lead to different clusters.
- Also, K Means algorithm is sensitive to noise. It may have stuck in local minima.

Machine Learning

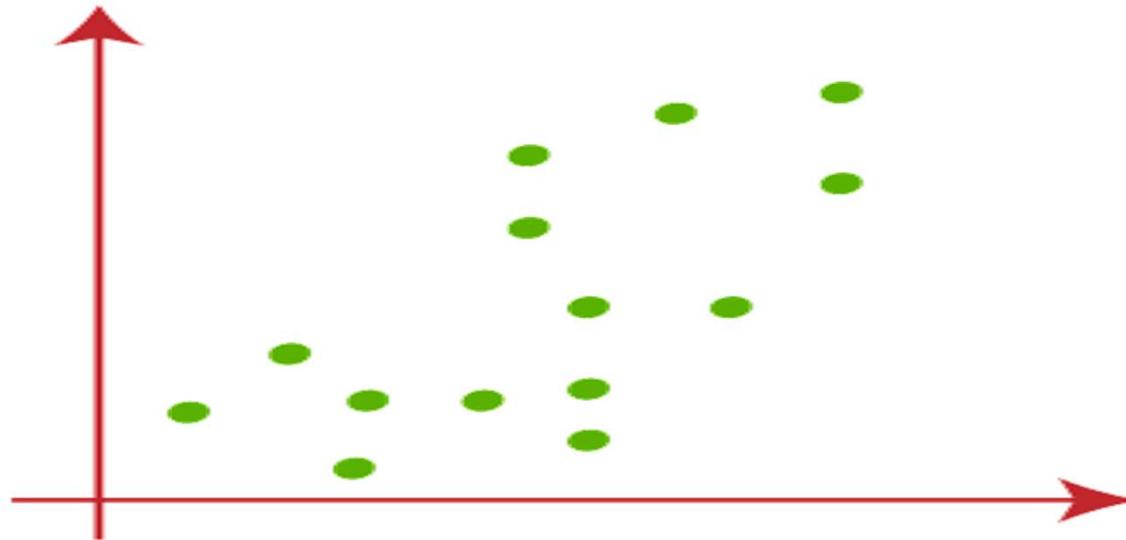
K-Means Clustering

How k-means clustering works?



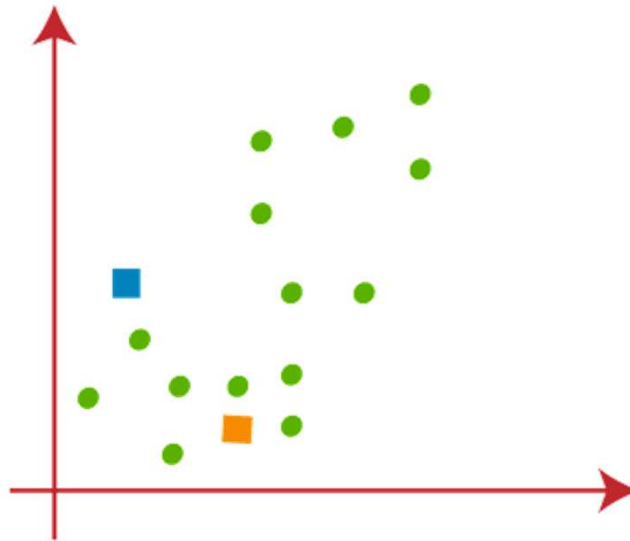
K-Means Clustering

How k-means clustering works?



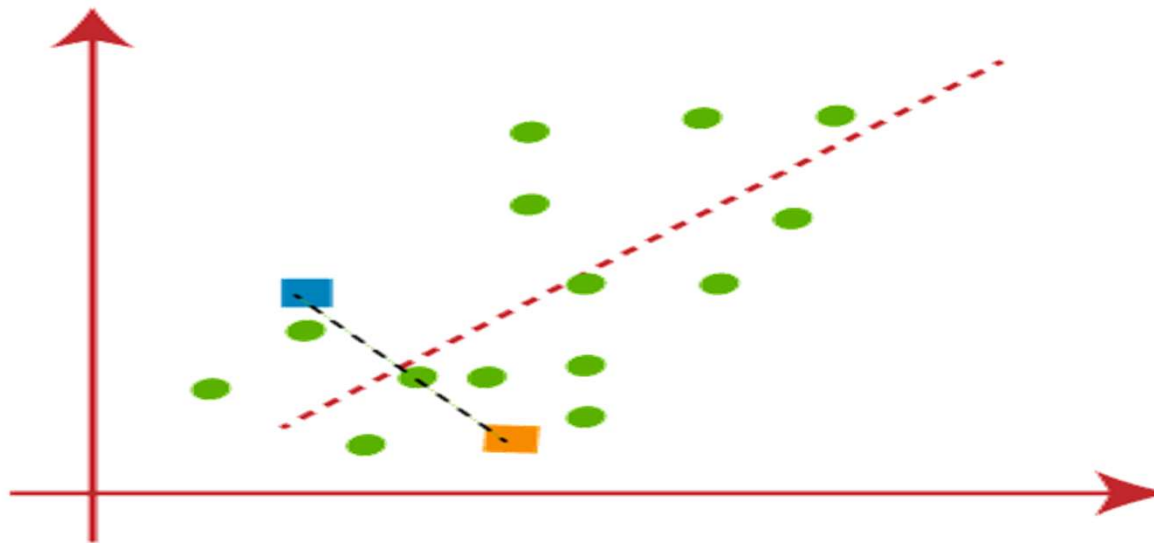
K-Means Clustering

How k-means clustering works?



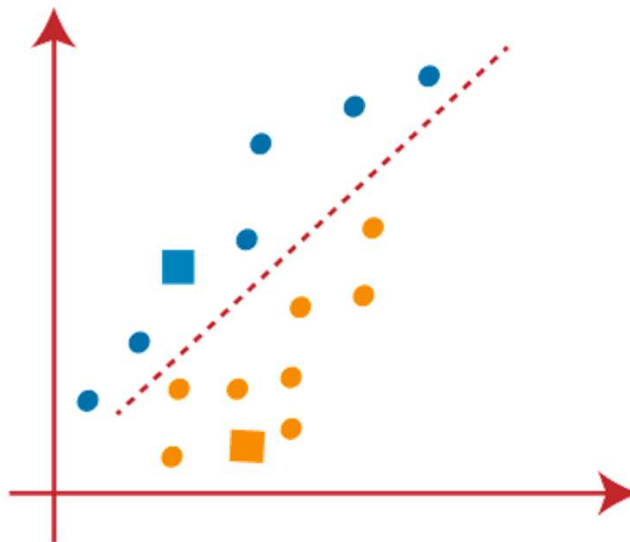
K-Means Clustering

How k-means clustering works?



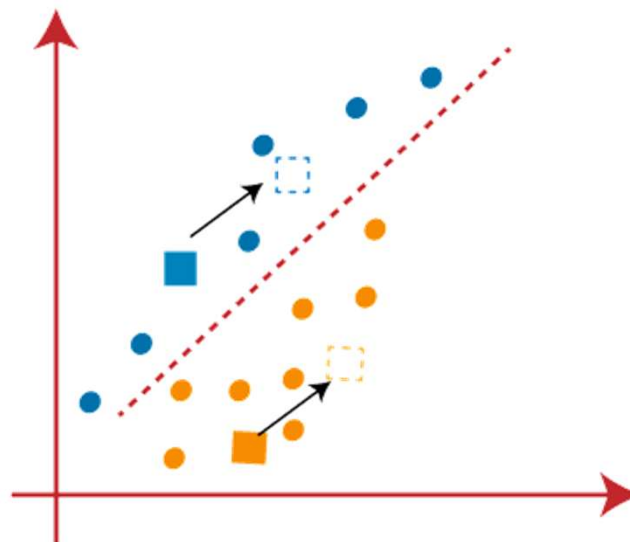
K-Means Clustering

How k-means clustering works?



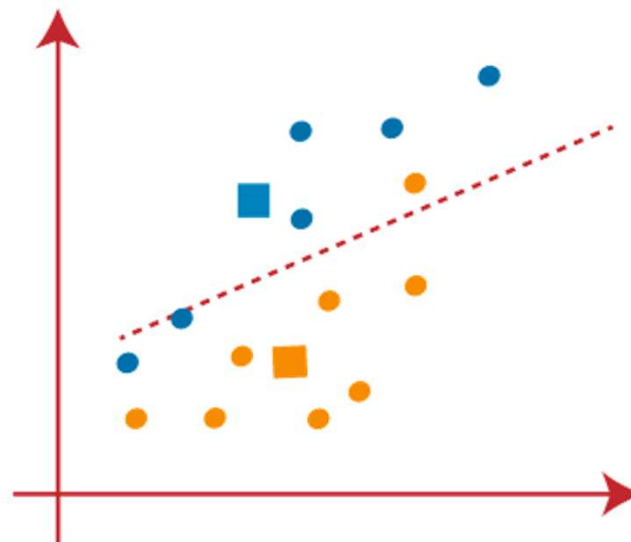
K-Means Clustering

How k-means clustering works?



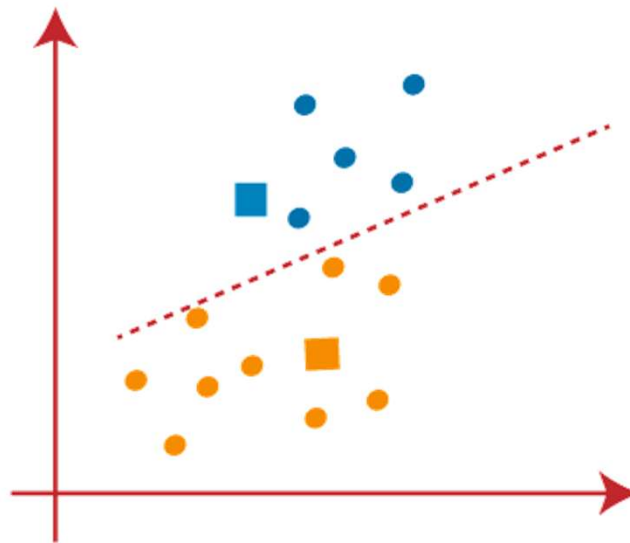
K-Means Clustering

How k-means clustering works?



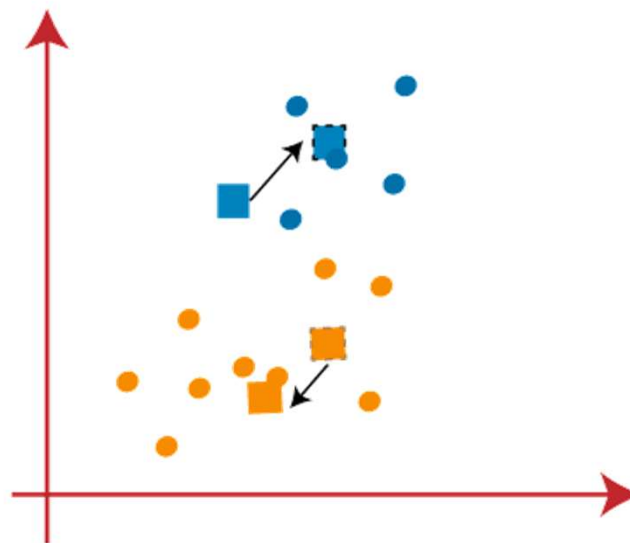
K-Means Clustering

How k-means clustering works?



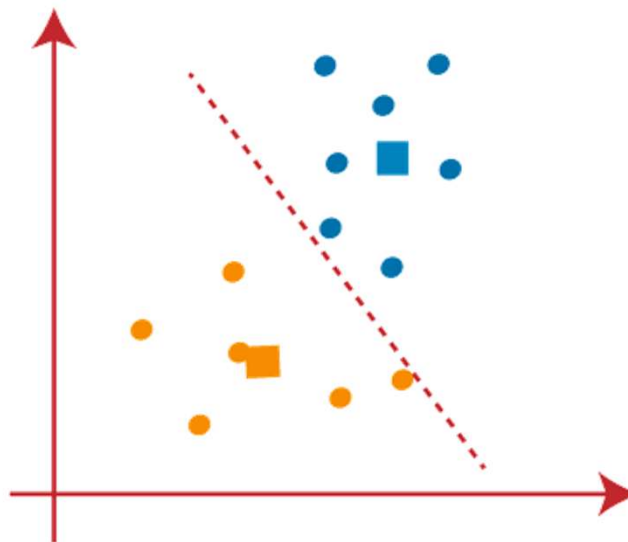
K-Means Clustering

How k-means clustering works?



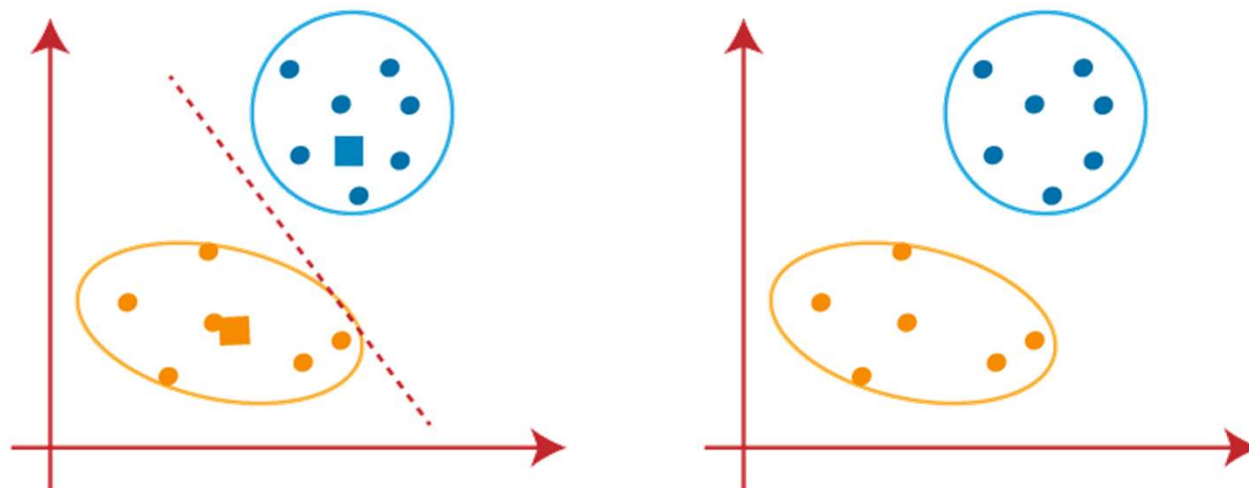
K-Means Clustering

How k-means clustering works?



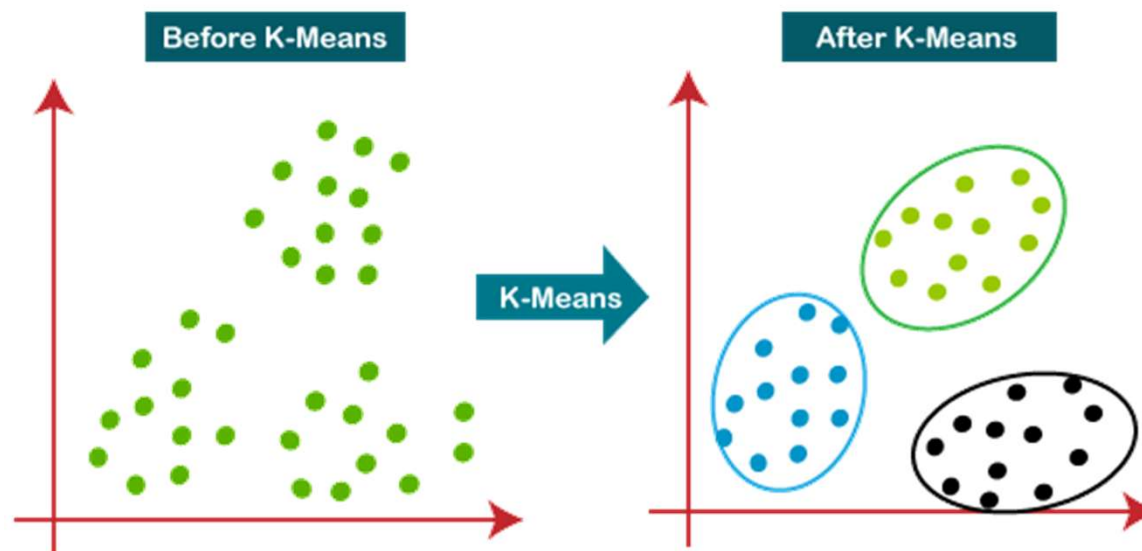
K-Means Clustering

How k-means clustering works?



K-Means Clustering

How k-means clustering works?



K-Means Clustering



How to choose the value of "K number of clusters" in K-means Clustering?

Elbow Method

The Elbow method is one of the most popular ways to find the optimal number of clusters. This method uses the concept of WCSS value. **WCSS** stands for **Within Cluster Sum of Squares**, which defines the total variations within a cluster. The formula to calculate the value of WCSS (for 3 clusters) is given below:

$$WCSS = \sum_{P_i \text{ in Cluster1}} \text{distance}(P_i C_1)^2 + \sum_{P_i \text{ in Cluster2}} \text{distance}(P_i C_2)^2 + \sum_{P_i \text{ in Cluster3}} \text{distance}(P_i C_3)^2$$

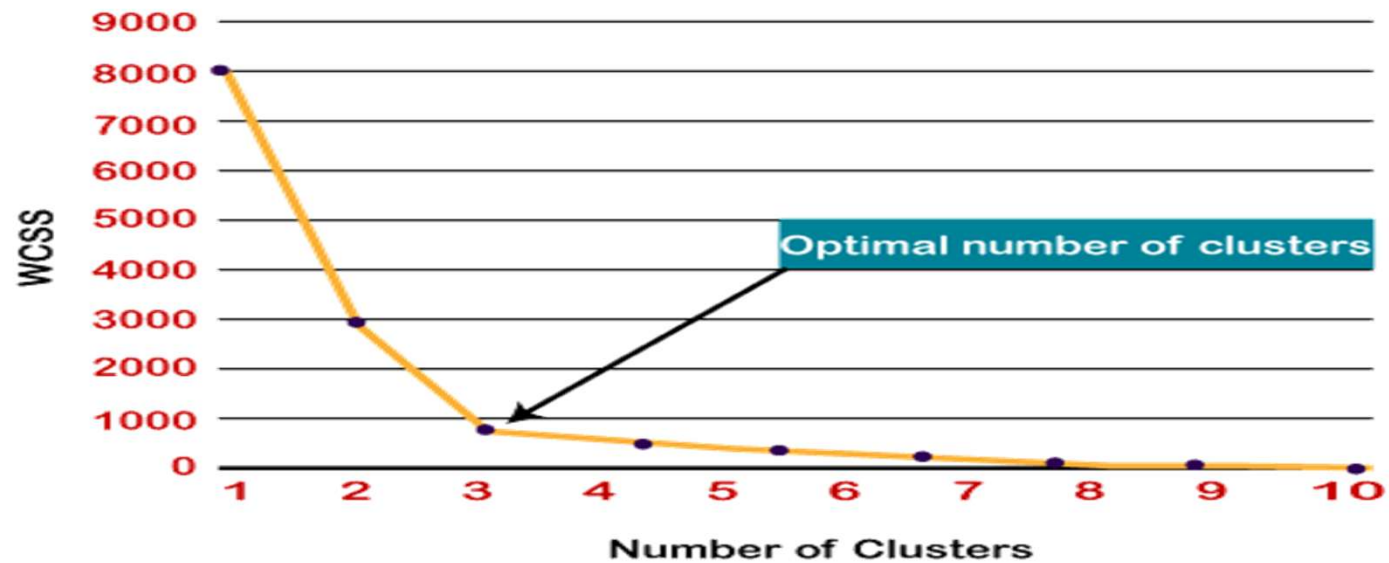
$\sum_{P_i \text{ in Cluster1}} \text{distance}(P_i C_1)^2$: It is the sum of the square of the distances between each data point and its centroid within a cluster1

K-Means Clustering



How to choose the value of "K number of clusters" in K-means Clustering?

Elbow Method

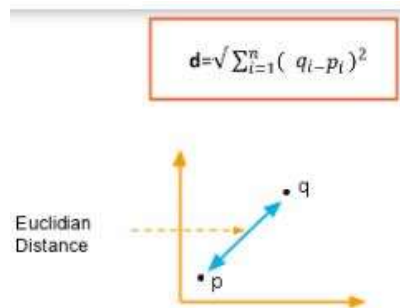


K-Means Clustering

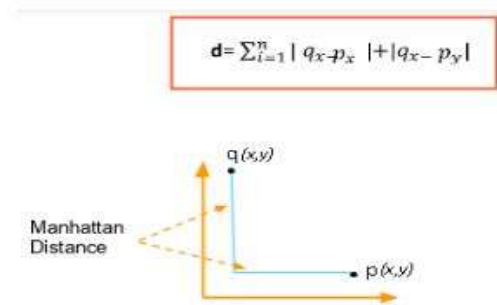
Distance



Euclidean distance measure



Manhattan distance measure

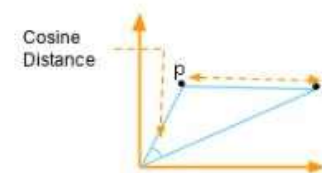


squared Euclidean distance measure

$$d = \sum_{i=1}^n (q_i - p_i)^2$$

Cosine distance measure

$$d = \frac{\sum_{i=0}^{n-1} q_i - p_x}{\sum_{i=0}^{n-1} (q_i)^2 \times \sum_{i=0}^{n-1} (p_i)^2}$$



K-Mode Clustering



- K-mode clustering is an unsupervised machine-learning technique used to group a set of data objects into a specified number of clusters, based on their categorical attributes.
- The algorithm is called “K-Mode” because it uses modes (i.e. the most frequent values) instead of means or medians to represent the clusters.
- In K-means clustering when we used categorical data after converting it into a numerical form. it doesn't give a good result for high-dimensional data.
- So, Some changes are made for categorical data

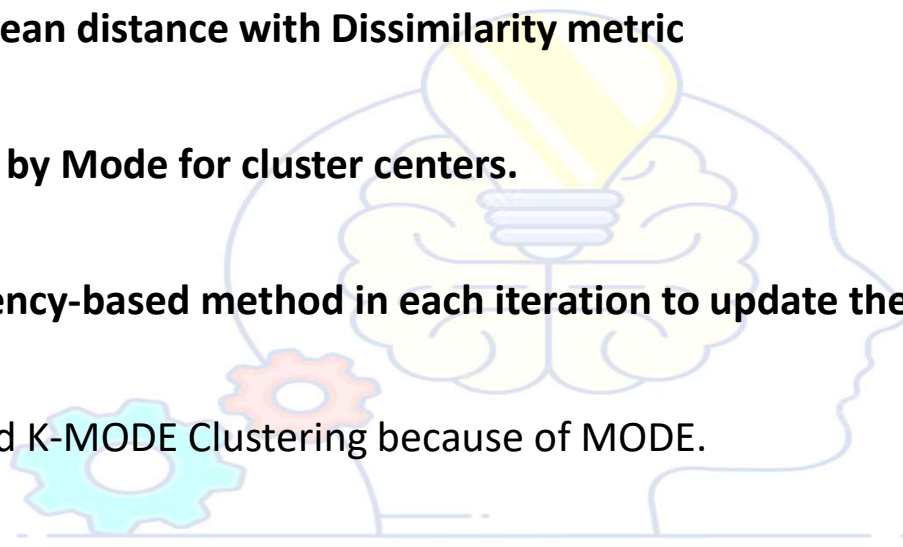
Machine Learning

K-Mode Clustering



- ❑ Replace Euclidean distance with Dissimilarity metric
- ❑ Replace Mean by Mode for cluster centers.
- ❑ Apply a frequency-based method in each iteration to update the mode.

➤ And then this is called K-MODE Clustering because of MODE.



Machine Learning

K-Mode Clustering



- Dissimilarity measurements between two data objects
- Let x and y be two categorical data objects defined by m features or attributes.

$$d(x, y) = \sum_{j=1}^m \delta(x_j, y_j)$$

$$\text{Where, } \delta(x_j, y_j) = \begin{cases} 0 & \text{if } x_j = y_j \\ 1 & \text{if } x_j \neq y_j \end{cases}$$

Machine Learning

K-Mode Clustering



- Dissimilarity measurements between two data objects
- Let x and y be two categorical data objects defined by m features or attributes.

$$d(x, y) = \sum_{j=1}^m \delta(x_j, y_j) \text{ Where, } \delta(x_j, y_j) = \begin{cases} 0 & \text{if } x_j = y_j \\ 1 & \text{if } x_j \neq y_j \end{cases}$$

S.No	Attribute 1	Attribute 2	Attribute 3
1	A	B	C
2	A	B	D
3	A	C	E
4	B	C	E

K Means Clustering – Numerical Example

Apply “**K-means clustering**” algorithm for the two-dimensional data _____ presented in Table 1. Find out the clusters for two iterations if the initial centroids: Centroid-1 is (75, 90) and Centroid-2 is (140, 130). Then give comments on your results.

Table 1

Data	A	B	C	D	E	F
X	75	80	140	145	120	125
Y	90	95	130	135	100	110

K Means Clustering – Numerical Example

Compute Euclidean Distance

The **Euclidean distance** between a point (X_i, Y_i) and a centroid (X_c, Y_c) is given by:

$$d = \sqrt{(X_i - X_c)^2 + (Y_i - Y_c)^2}$$

We compute distances for each data point:

Data	(75,90) Distance	(140,130) Distance	Cluster
A (75,90)	0.00	76.12	C1
B (80,95)	7.07	71.12	C1
C (140,130)	76.12	0.00	C2
D (145,135)	82.31	7.07	C2
E (120,100)	46.10	29.15	C2
F (125,110)	53.15	18.03	C2

K Means Clustering – Numerical Example

Data	(75,90) Distance	(140,130) Distance	Cluster
A (75,90)	0.00	76.12	C1
B (80,95)	7.07	71.12	C1
C (140,130)	76.12	0.00	C2
D (145,135)	82.31	7.07	C2
E (120,100)	46.10	29.15	C2
F (125,110)	53.15	18.03	C2

Cluster Assignment after First Iteration

- Cluster-1 (C1): A (75,90), B (80,95)
- Cluster-2 (C2): C (140,130), D (145,135), E (120,100), F (125,110)

K Means Clustering – Numerical Example

Compute New Centroids

New centroids are calculated as the mean of the points in each cluster:

- New Centroid-1 (C1):

$$X_{\text{new}} = \frac{(75 + 80)}{2} = 77.5, \quad Y_{\text{new}} = \frac{(90 + 95)}{2} = 92.5$$

(77.5, 92.5)

- New Centroid-2 (C2):

$$X_{\text{new}} = \frac{(140 + 145 + 120 + 125)}{4} = 132.5, \quad Y_{\text{new}} = \frac{(130 + 135 + 100 + 110)}{4} = 118.75$$

(132.5, 118.75)

K Means Clustering – Numerical Example

We now compute new distances using these centroids.

Data	(77.5,92.5) Distance	(132.5,118.75) Distance	Cluster
A (75,90)	3.54	61.87	C1
B (80,95)	3.54	56.99	C1
C (140,130)	66.08	10.18	C2
D (145,135)	74.41	20.42	C2
E (120,100)	43.86	14.71	C2
F (125,110)	50.12	9.47	C2

Cluster Assignment after Second Iteration

- **Cluster-1 (C1):** A (75,90), B (80,95)
- **Cluster-2 (C2):** C (140,130), D (145,135), E (120,100), F (125,110)

K Means Clustering – Numerical Example

Cluster Assignment after Second Iteration

- Cluster-1 (C1): A (75,90), B (80,95)
- Cluster-2 (C2): C (140,130), D (145,135), E (120,100), F (125,110)

Since the cluster assignments **did not change**, the algorithm **converged** after the second iteration.

- New Centroid-1 (C1):

$$X_{\text{new}} = \frac{(75 + 80)}{2} = 77.5, \quad Y_{\text{new}} = \frac{(90 + 95)}{2} = 92.5$$

(77.5, 92.5)

- New Centroid-2 (C2):

$$X_{\text{new}} = \frac{(140 + 145 + 120 + 125)}{4} = 132.5, \quad Y_{\text{new}} = \frac{(130 + 135 + 100 + 110)}{4} = 118.75$$

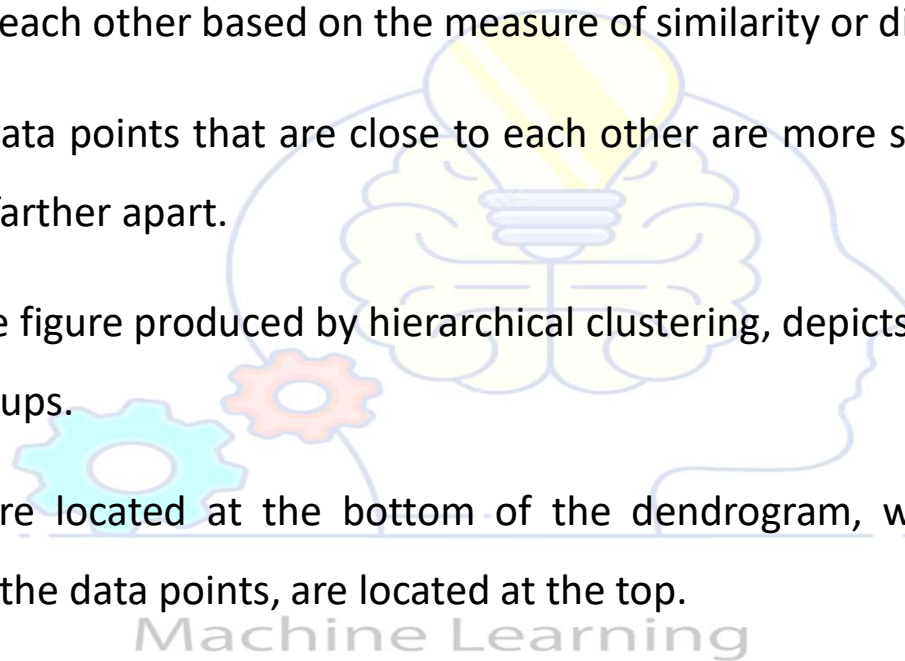
(132.5, 118.75)

This means the algorithm **found stable centroids**, and further iterations will keep the clusters the same.

Hierarchical Clustering



- Hierarchical clustering is a connectivity-based clustering model that groups the data points together that are close to each other based on the measure of similarity or distance.
- The assumption is that data points that are close to each other are more similar or related than data points that are farther apart.
- A dendrogram, a tree-like figure produced by hierarchical clustering, depicts the hierarchical relationships between groups.
- Individual data points are located at the bottom of the dendrogram, while the largest clusters, which include all the data points, are located at the top.



Hierarchical Clustering



- In order to generate different numbers of clusters, the dendrogram can be sliced at various heights.
- The dendrogram is created by iteratively merging or splitting clusters based on a measure of similarity or distance between data points.
- Clusters are divided or merged repeatedly until all data points are contained within a single cluster, or until the predetermined number of clusters is attained.
- The dendrogram can be sliced at this height to determine the number of clusters.

Machine Learning

Hierarchical Clustering



➤ Basically, there are two types of hierarchical Clustering:

- ❑ Agglomerative Clustering

- ❑ Divisive clustering

(Or)

- ❑ AGNES- **AG**glomerative **NE**sting Hierarchical clustering algorithm

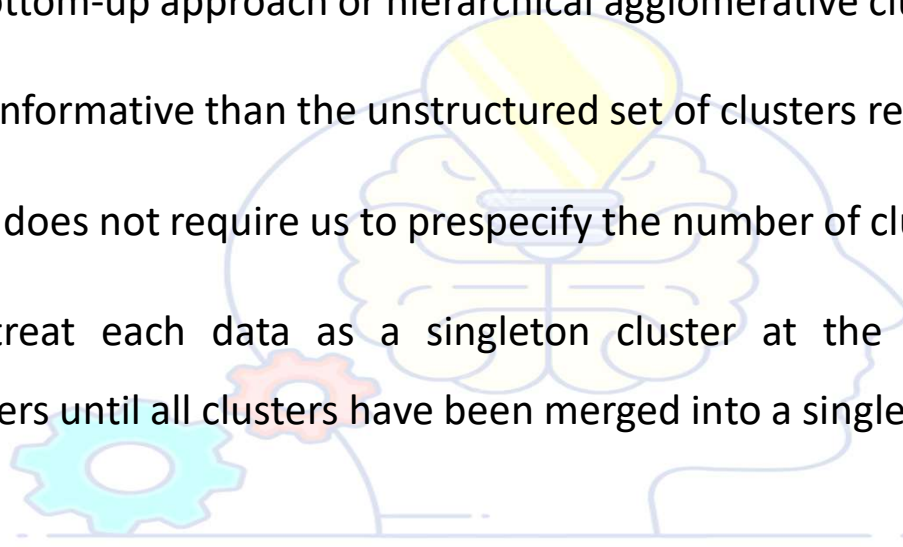
- ❑ DIANA- **DI**vise **ANA**lysis Clustering Algorithm

Machine Learning

Hierarchical Clustering - AGNES



- It is also known as the bottom-up approach or hierarchical agglomerative clustering (HAC).
- A structure that is more informative than the unstructured set of clusters returned by flat clustering.
- This clustering algorithm does not require us to prespecify the number of clusters.
- Bottom-up algorithms treat each data as a singleton cluster at the outset and then successively agglomerate pairs of clusters until all clusters have been merged into a single cluster that contains all data.

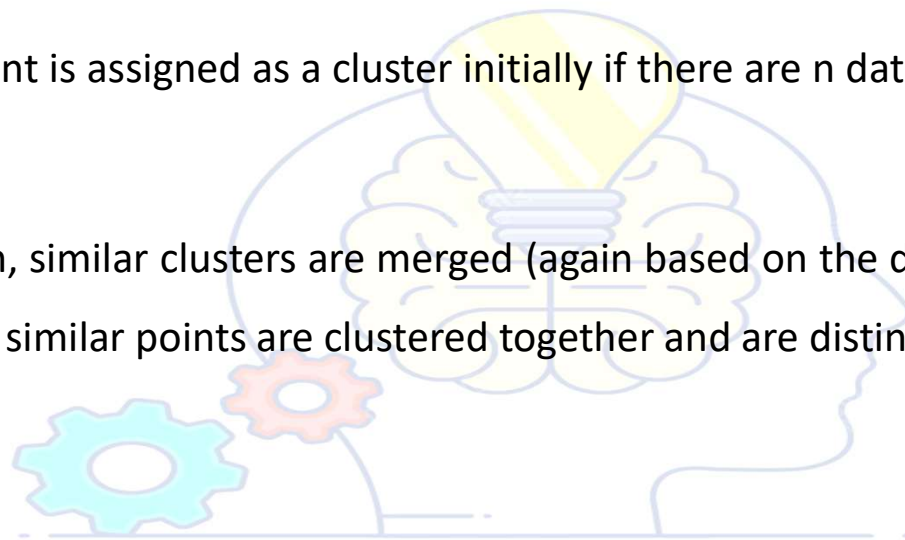


Machine Learning

Hierarchical Clustering - AGNES

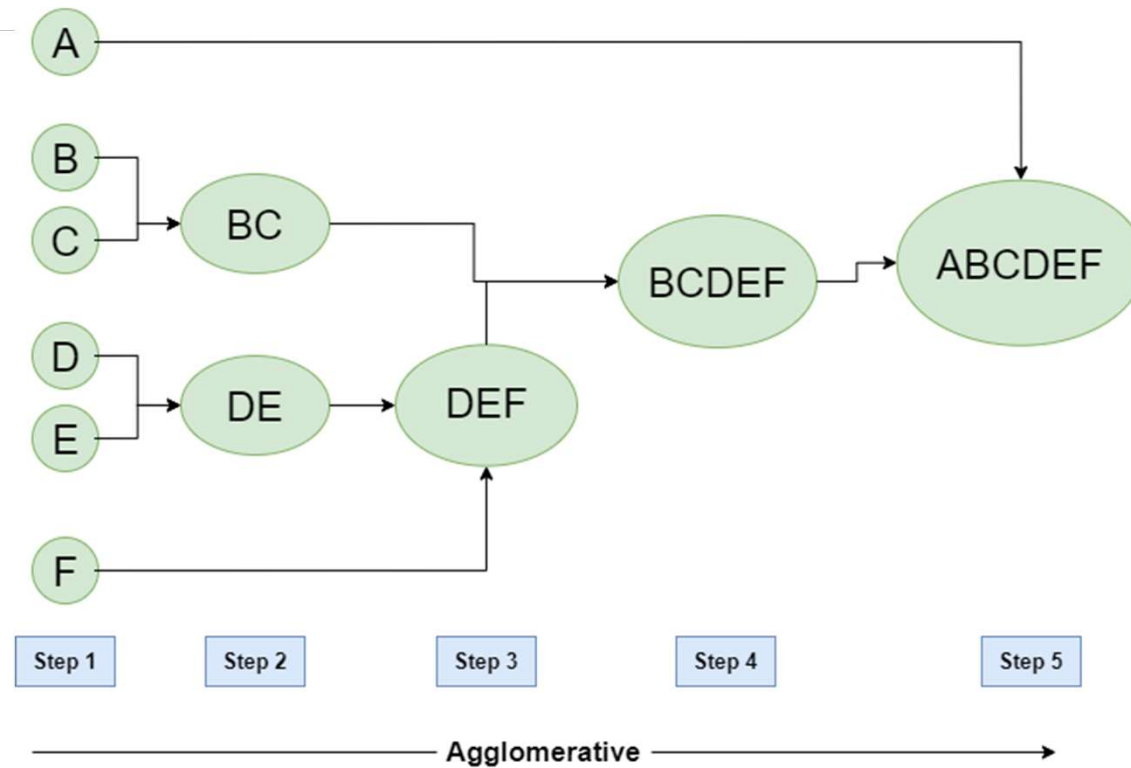


- It is an inside-out or bottoms-up approach.
- Here every data point is assigned as a cluster initially if there are n data points n clusters will be formed initially.
- In the next iteration, similar clusters are merged (again based on the density and distances), this continuous until similar points are clustered together and are distinct for other clusters.



Machine Learning

Hierarchical Clustering - AGNES



Hierarchical Clustering - AGNES

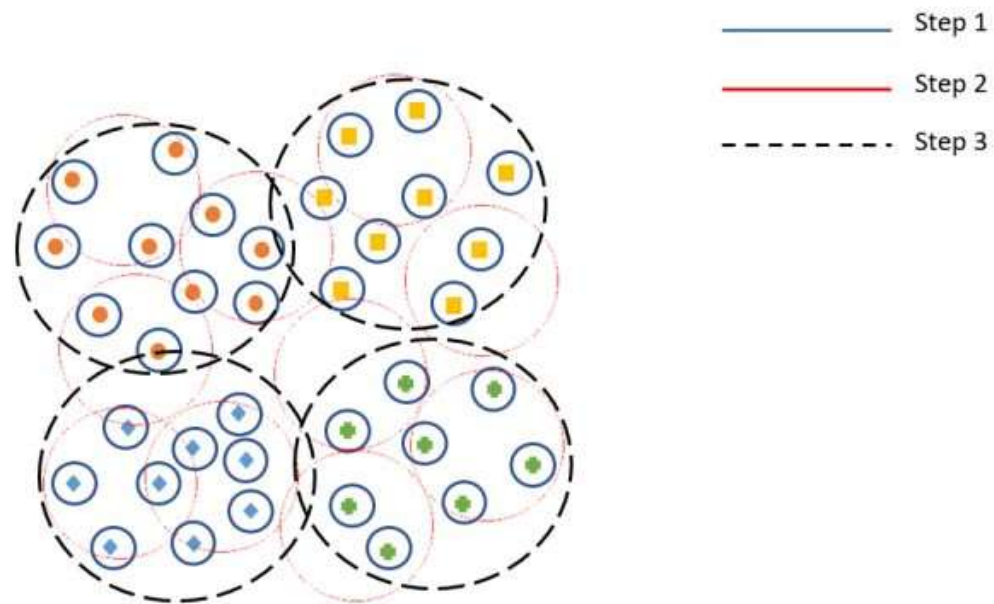


Steps:

- Consider each alphabet as a single cluster and calculate the distance of one cluster from all the other clusters.
- In the second step, comparable clusters are merged together to form a single cluster. Let's say cluster (B) and cluster (C) are very similar to each other therefore we merge them in the second step similarly to cluster (D) and (E) and at last, we get the clusters [(A), (BC), (DE), (F)]
- We recalculate the proximity according to the algorithm and merge the two nearest clusters([(DE), (F)]) together to form new clusters as [(A), (BC), (DEF)]
- Repeating the same process; The clusters DEF and BC are comparable and merged together to form a new cluster. We're now left with clusters [(A), (BCDEF)].
- At last, the two remaining clusters are merged together to form a single cluster [(ABCDEF)].

Machine Learning

Hierarchical Clustering - AGNES

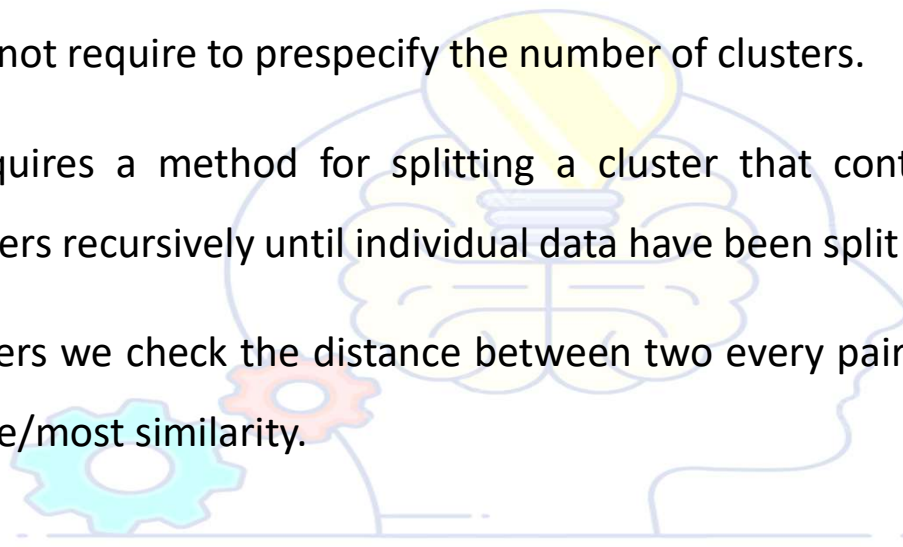


Images subject to copyright: The Datum

Hierarchical Clustering - DIANA



- It is also known as a top-down approach.
- This algorithm also does not require to prespecify the number of clusters.
- Top-down clustering requires a method for splitting a cluster that contains the whole data and proceeds by splitting clusters recursively until individual data have been split into singleton clusters.
- While merging two clusters we check the distance between two every pair of clusters and merge the pair with the least distance/most similarity.



Machine Learning

Hierarchical Clustering - DIANA

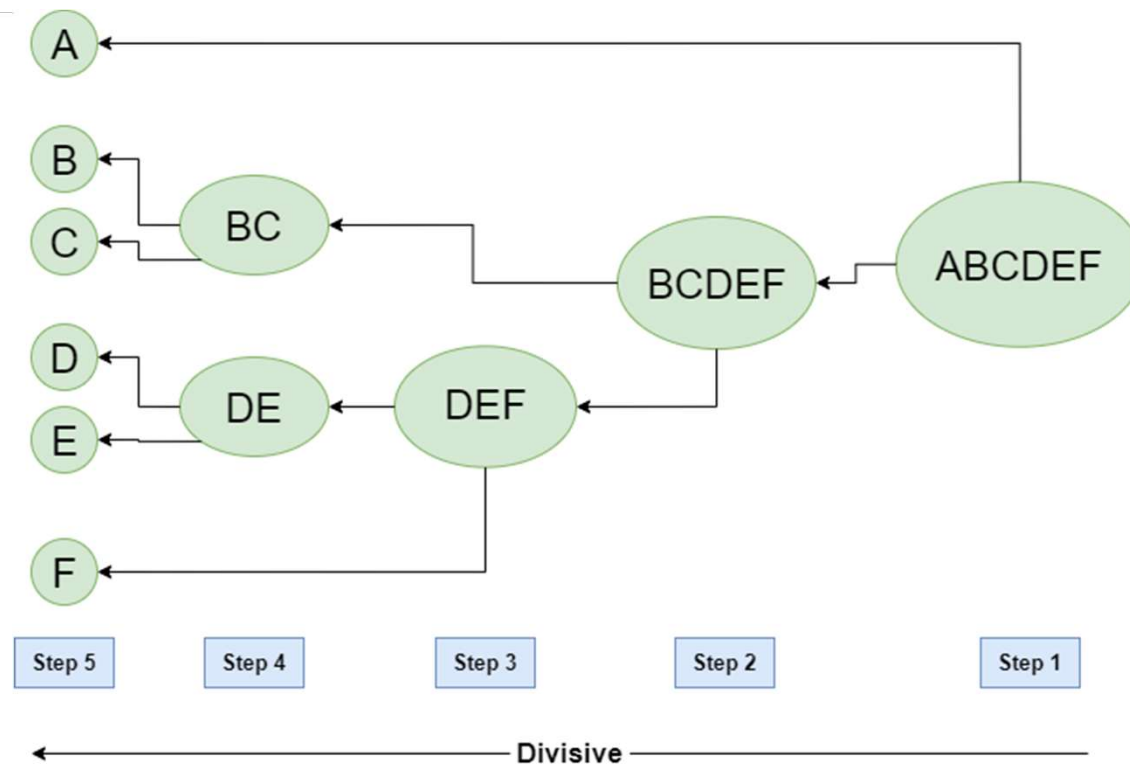


➤ But the question is how is that distance determined. There are different ways of defining Inter Cluster distance/similarity. Some of them are:

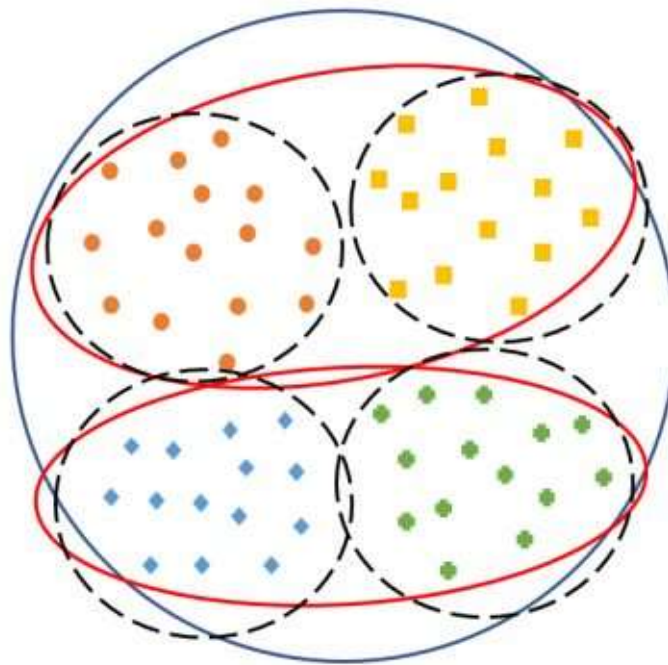
- **Min Distance:** Find the minimum distance between any two points of the cluster.
- **Max Distance:** Find the maximum distance between any two points of the cluster.
- **Group Average:** Find the average distance between every two points of the clusters.
- **Ward's Method:** The similarity of two clusters is based on the increase in squared error when two clusters are merged.

Machine Learning

Hierarchical Clustering - DIANA



Hierarchical Clustering - DIANA



- Step 1
- Step 2
- Step 3

Images subject to copyright: The Datum

Hierarchical Clustering – AGNES Vs DIANA



- **Divisive clustering is more complex** as compared to agglomerative clustering, as in the case of divisive clustering we need a flat clustering method as “subroutine” to split each cluster until we have each data having its own singleton cluster.
- **Divisive clustering is more efficient** if we do not generate a complete hierarchy all the way down to individual data leaves. The time complexity of a naive agglomerative clustering is $O(n^3)$ because we exhaustively scan the $N \times N$ matrix `dist_mat` for the lowest distance in each of $N-1$ iterations. Using priority queue data structure we can reduce this complexity to $O(n^2 \log n)$. By using some more optimizations, it can be brought down to $O(n^2)$.

Hierarchical Clustering – AGNES Vs DIANA



- Whereas for divisive clustering given a fixed number of top levels, using an efficient flat algorithm like K-Means, divisive algorithms are linear in the number of patterns and clusters.
- **A divisive algorithm is also more accurate.** Agglomerative clustering makes decisions by considering the local patterns or neighbor points without initially taking into account the global distribution of data. These early decisions cannot be undone. whereas divisive clustering takes into consideration the global distribution of data when making top-level partitioning decisions.

Machine Learning

Gaussian Mixture Model



Machine Learning

Gaussian Mixture Model



- Clustering is a key technique in unsupervised learning, used to group similar data points together.
- While traditional methods like K-Means and Hierarchical Clustering are widely used, they assume that clusters are well-separated and have rigid shapes.
- This can be limiting in real-world scenarios where clusters can be more complex.
- To overcome these limitations, **Gaussian Mixture Models (GMM)** offer a more flexible approach.
- Unlike K-Means, which assigns each point to a single cluster, GMM uses a probabilistic approach to cluster the data, allowing clusters to have more varied shapes and soft boundaries.

Gaussian Mixture Model Vs K-Means



- A **Gaussian mixture model** is a **soft clustering technique** used in unsupervised learning to determine the probability that a given data point belongs to a cluster. It's composed of several Gaussians, each identified by $k \in \{1, \dots, K\}$, where K is the number of clusters in a data set and is comprised of the following parameters.
- **K-means** is a clustering algorithm that assigns each data point to one cluster based on the closest centroid. It's a **hard clustering method**, meaning each point belongs to only one cluster with no uncertainty.
- ***Gaussian Mixture Models (GMM)** use soft clustering, where data points can belong to multiple clusters with a certain probability. This provides a more flexible and nuanced way to handle clusters, especially when points are close to multiple centroids.

Gaussian Mixture Model



➤ Unlike K-Means, where the clustering process relies solely on the centroid and assigns each data point to one cluster, GMM uses a probabilistic approach. Here's how GMM performs clustering:

1. Multiple Gaussians (Clusters): Each cluster is represented by a Gaussian distribution, and the data points are assigned probabilities of belonging to different clusters based on their distance from each Gaussian.

2. Parameters of a Gaussian: The core of GMM is made up of three main parameters for each Gaussian:

- Mean (μ): The center of the Gaussian distribution.
- Covariance (Σ): Describes the spread or shape of the cluster.
- Mixing Probability (π): Determines how dominant or likely each cluster is in the data.

Gaussian Mixture Model



➤ The probability of data point coming from Gaussian cluster k is expressed as

$$P(z_n = k \mid x_n) = \frac{\pi_k \cdot \mathcal{N}(x_n \mid \mu_k, \Sigma_k)}{\sum_{k=1}^K \pi_k \cdot \mathcal{N}(x_n \mid \mu_k, \Sigma_k)}$$

where,

- $z_n=k$ is a latent variable indicating which Gaussian the point belongs to.
- π_k is the mixing probability of the k -th Gaussian.
- $\mathcal{N}(x_n \mid \mu_k, \Sigma_k)$ is the Gaussian distribution with mean μ_k and covariance Σ_k

Machine Learning

Gaussian Mixture Model

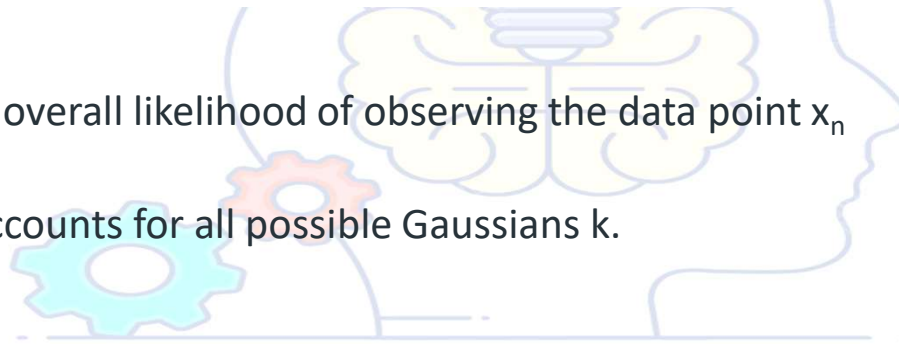


➤ The overall likelihood of observing a data point x_n under all Gaussians is achieved by summing over all possible clusters (Gaussians) for each point:

$$P(x_n) = \sum_{k=1}^K \pi_k \mathcal{N}(x_n | \mu_k, \Sigma_k)$$

where,

- $P(x_n)$ is the overall likelihood of observing the data point x_n
- The sum accounts for all possible Gaussians k .

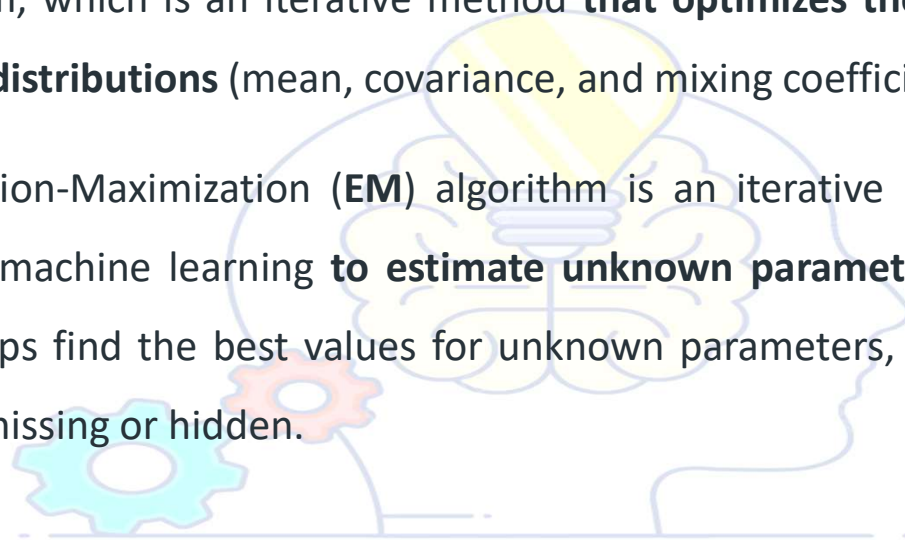


Machine Learning

Expectation-Maximization (EM) Algorithm



- To fit a Gaussian Mixture Model to the data, use the Expectation-Maximization (EM) algorithm, which is an iterative method **that optimizes the parameters of the Gaussian distributions** (mean, covariance, and mixing coefficients).
- The Expectation-Maximization (EM) algorithm is an iterative method used in unsupervised machine learning **to estimate unknown parameters in statistical models**. It helps find the best values for unknown parameters, especially when some data is missing or hidden.



Machine Learning

Expectation-Maximization (EM) Algorithm



➤ It works in two steps:

1. Expectation Step (E-step): In this step, the algorithm calculates the probability that each data point belongs to each cluster based on the current parameter estimates (mean, covariance, mixing coefficients).

2. Maximization Step (M-step): After estimating the probabilities, the algorithm updates the parameters (mean, covariance, and mixing coefficients) to better fit the data.

These two steps are repeated until the model converges, meaning the parameters no longer change significantly between iterations.

Expectation-Maximization (EM) Algorithm



➤ This process repeats until the model reaches a stable solution, **improving accuracy with each iteration**. EM is widely used in clustering (e.g., Gaussian Mixture Models) and handling missing data

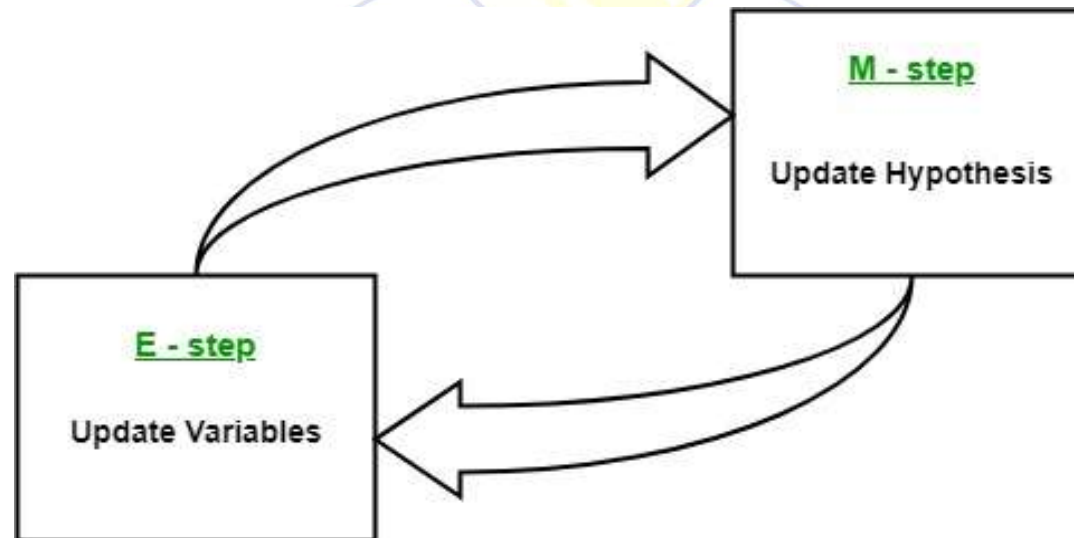
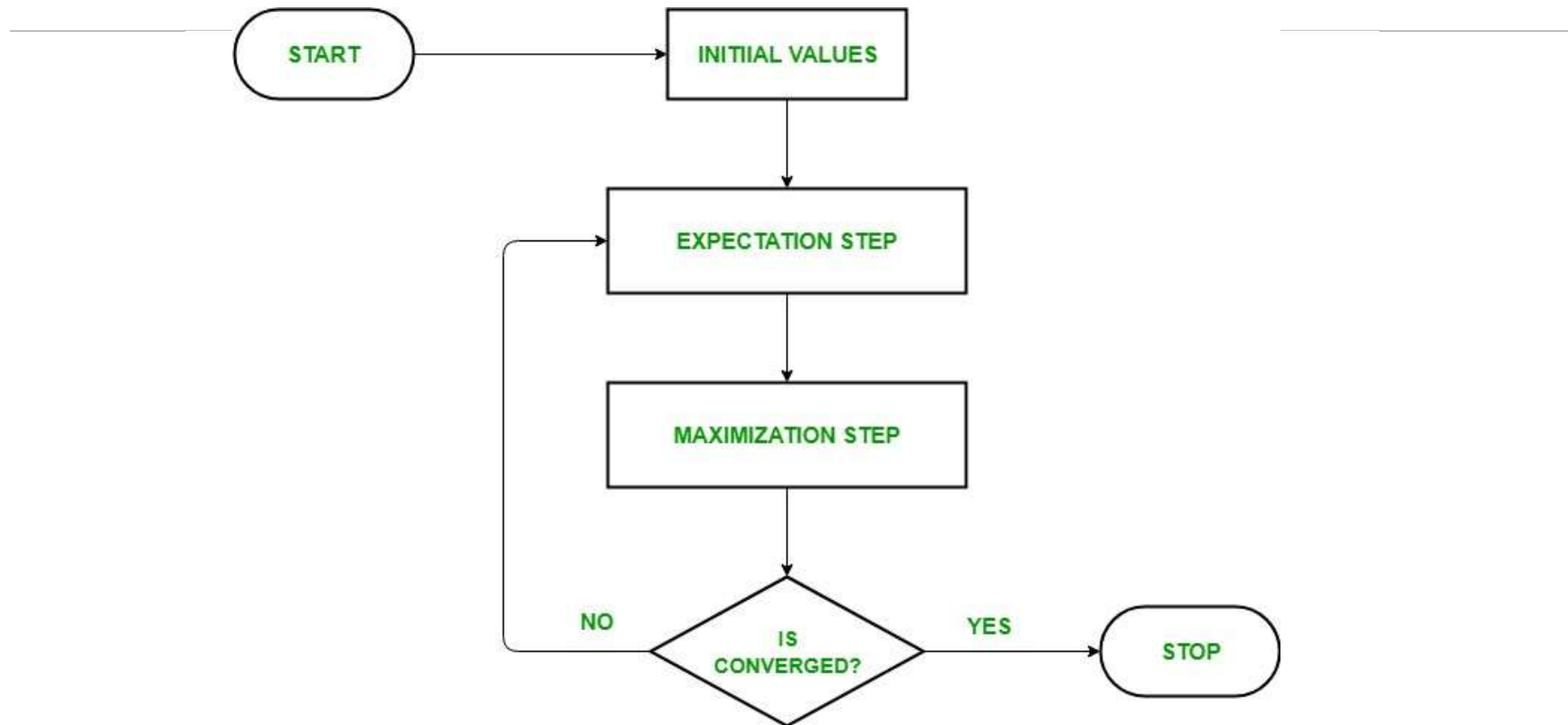


Fig: Expectation-Maximization in EM Algorithm

Expectation-Maximization (EM) Algorithm



Expectation-Maximization (EM) Algorithm



1. Initialization:

The algorithm starts with initial parameter values and assumes the observed data comes from a specific model.

2. E-Step (Expectation Step):

- ❑ Estimate the missing or hidden data based on the current parameters.
- ❑ Calculate the posterior probability (responsibility) of each latent variable given the observed data.
- ❑ Compute the log-likelihood of the observed data using the current parameter estimates.

Expectation-Maximization (EM) Algorithm



3. M-Step (Maximization Step):

- ❑ Update the model parameters by maximizing the log-likelihood computed in the E-step.
- ❑ This involves solving an optimization problem to find parameter values that improve the model fit.

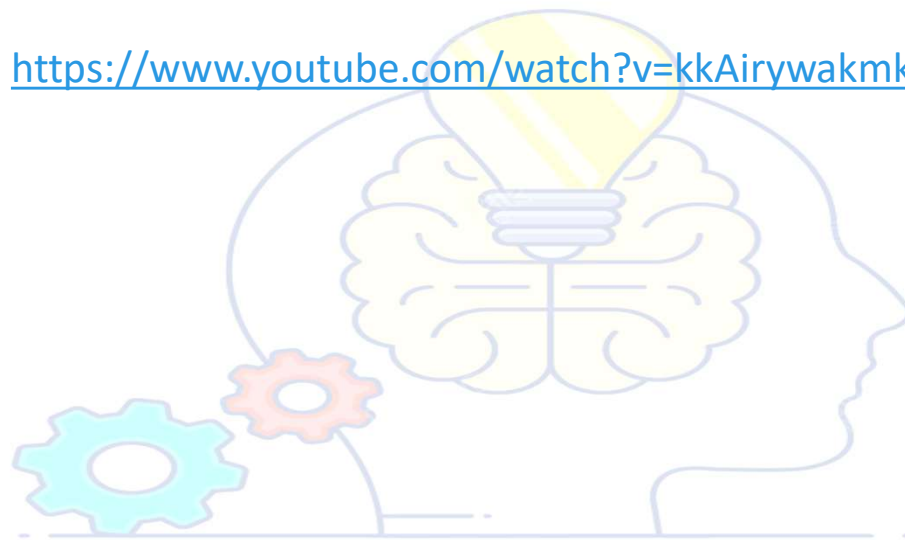
4. Convergence:

- ❑ Check if the model parameters are stable (converging).
- ❑ If the changes in log-likelihood or parameters are below a set threshold, stop. If not, repeat the E-step and M-step until convergence is reached

Gaussian Mixture Model

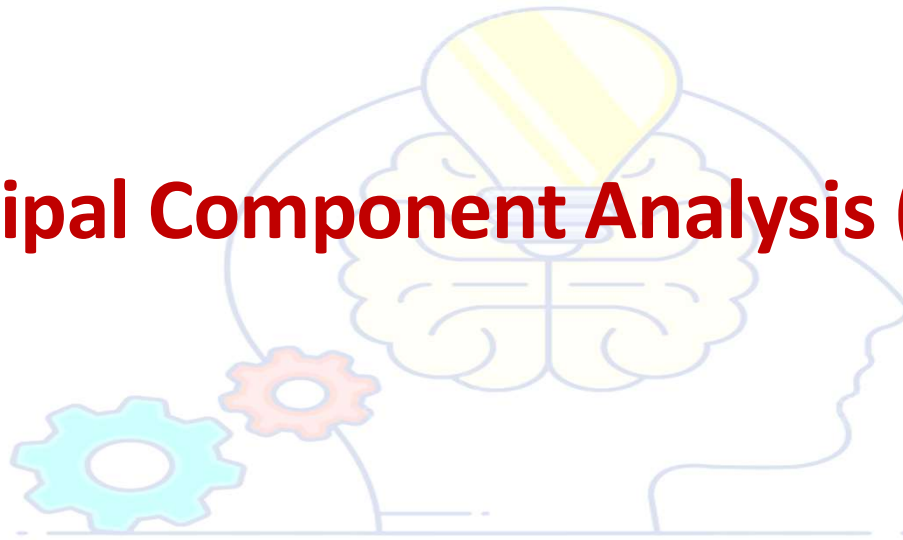


<https://www.youtube.com/watch?v=kkAirywakmk>



Machine Learning

Principal Component Analysis (PCA)



Machine Learning

Principal Component Analysis (PCA)



- ❑ Principal component analysis, or PCA, is a dimensionality reduction method that is often used to reduce the dimensionality of large data sets, by transforming a large set of variables into a smaller one that still contains most of the information in the large set.
- ❑ Reducing the number of variables of a data set naturally comes at the expense of accuracy, but the trick in dimensionality reduction is to trade a little accuracy for simplicity.
- ❑ Because smaller data sets are easier to explore and visualize, and thus make analyzing data points much easier and faster for machine learning algorithms without extraneous variables to process.

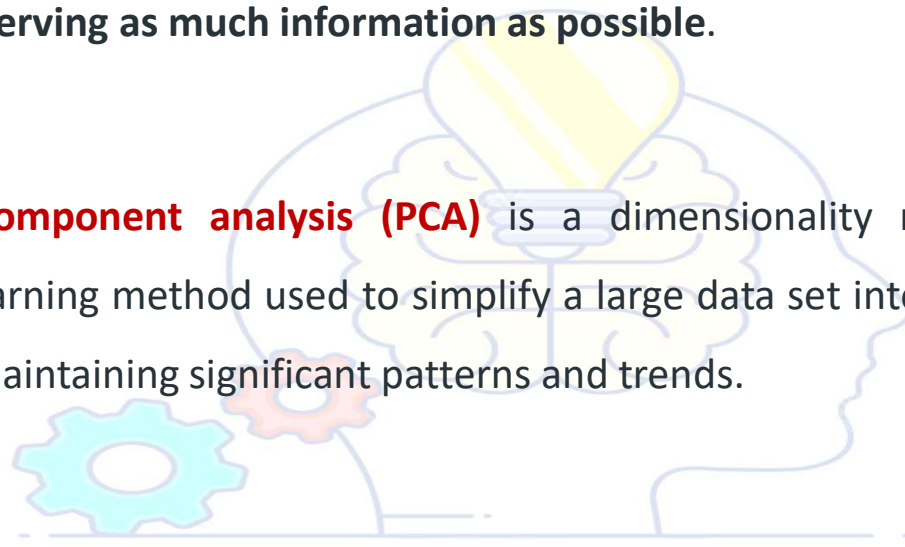
Principal Component Analysis (PCA)



□ The idea of PCA is simple: **reduce the number of variables of a data set, while preserving as much information as possible.**

□ Definition:

Principal component analysis (PCA) is a dimensionality reduction and machine learning method used to simplify a large data set into a smaller set while still maintaining significant patterns and trends.

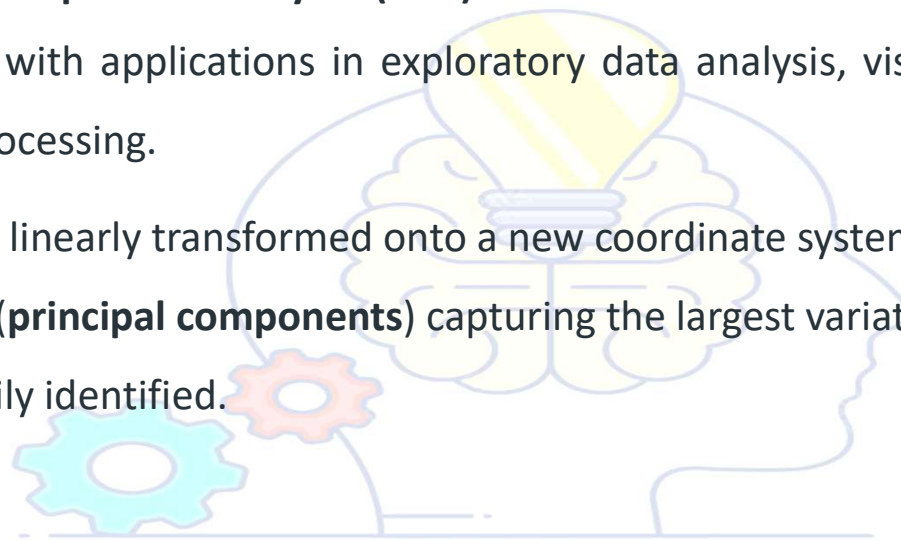


Machine Learning

Principal Component Analysis (PCA)



- ❑ **Principal component analysis (PCA)** is a **linear dimensionality reduction technique** with applications in exploratory data analysis, visualization and data preprocessing.
- ❑ The data is linearly transformed onto a new coordinate system such that the directions (**principal components**) capturing the largest variation in the data can be easily identified.



Machine Learning

Principal Component Analysis (PCA)

what is Principal Components?



- ❑ **Principal components** are new variables that are constructed as linear combinations or mixtures of the initial variables.
- ❑ These combinations are done in such a way that the new variables (i.e., principal components) are uncorrelated and most of the information within the initial variables is squeezed or compressed into the first components.
- ❑ So, the idea is 10-dimensional data gives you 10 principal components, but PCA tries to put maximum possible information in the first component, then maximum remaining information in the second and so on

Machine Learning

A stylized graphic featuring a profile of a human head with a brain. Inside the brain, there are circuit-like patterns and a central yellow circle. The head is surrounded by various icons representing data, technology, and learning, such as a lightbulb, a gear, a magnifying glass, and a plus sign. The text "MACHINE LEARNING" is written in a bold, sans-serif font at the bottom of the graphic.



Principal Component Analysis (PCA)

what is Principal Components?



- ❑ Organizing information in principal components this way will **allow to reduce dimensionality without losing much information**, and this by discarding the components with low information and considering the remaining components as new variables.
- ❑ Principal components are less interpretable and don't have any real meaning since they are constructed as linear combinations of the initial variables.
- ❑ **Geometrically speaking, principal components represent the directions of the data that explain a maximal amount of variance**, that is to say, the lines that capture most information of the data.

Principal Component Analysis (PCA)

what is Principal Components?



- ❑ The relationship between variance and information here, is that, the larger the variance carried by a line, the larger the dispersion of the data points along it, and the larger the dispersion along a line, the more information it has.
- ❑ Simply, just think of **principal components as new axes that provide the best angle to see and evaluate the data**, so that the differences between the observations are better visible.
- ❑ As there are as many principal components as there are variables in the data, principal components are constructed in such a manner that the first principal component accounts for the largest possible variance in the data set.

Principal Component Analysis (PCA)

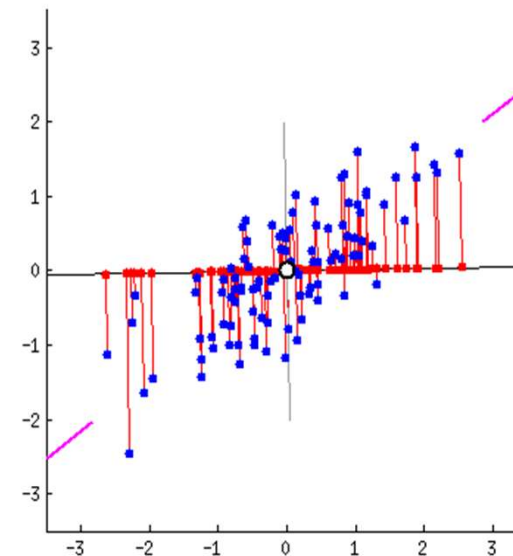
How PCA Constructs the Principal Components

□ let's assume that the scatter plot of our

data set is as shown below, can we guess

the first principal component ? Yes, it's approximately the line that matches the purple marks because it goes through the origin and it's the line in which the projection of the points (red dots) is the most spread out.

□ Or **mathematically speaking, it's the line that maximizes the variance (the average of the squared distances from the projected points (red dots) to the origin).**

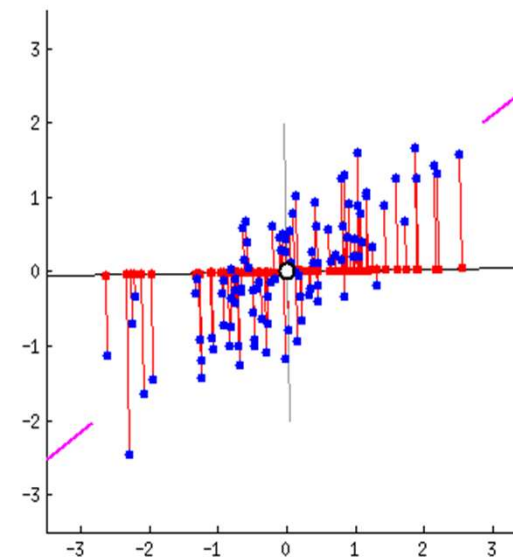


Principal Component Analysis (PCA)

How PCA Constructs the Principal Components



- The second principal component is calculated in the same way, with the condition that it is uncorrelated with (i.e., perpendicular to) the first principal component and that it accounts for the next highest variance.
- This continues until a total of p principal components have been calculated, equal to the original number of variables.



Machine Learning



Principal Component Analysis (PCA)

How Do You Do a Principal Component Analysis? – Steps involved

1. **Standardize** the range of continuous initial variables
2. Compute the **covariance matrix** to identify correlations
3. Compute the **eigenvectors and eigenvalues** of the covariance matrix to identify the principal components
4. Create a **feature vector** to decide which principal components to keep
5. **Recast the data** along the principal components' axes

Machine Learning

Principal Component Analysis (PCA)

Step-by-Step Explanation

Step 1: Standardization



- ❑ The aim of this step is to standardize the range of the continuous initial variables so that each one of them contributes equally to the analysis.
- ❑ More specifically, the reason why it is critical to perform standardization prior to PCA, is that the latter is quite sensitive regarding the variances of the initial variables.
- ❑ That is, if there are large differences between the ranges of initial variables, those variables with larger ranges will dominate over those with small ranges (for example, a variable that ranges between 0 and 100 will dominate over a variable that ranges between 0 and 1), which will lead to biased results. So, transforming the data to comparable scales can prevent this problem.

Principal Component Analysis (PCA)

Step-by-Step Explanation

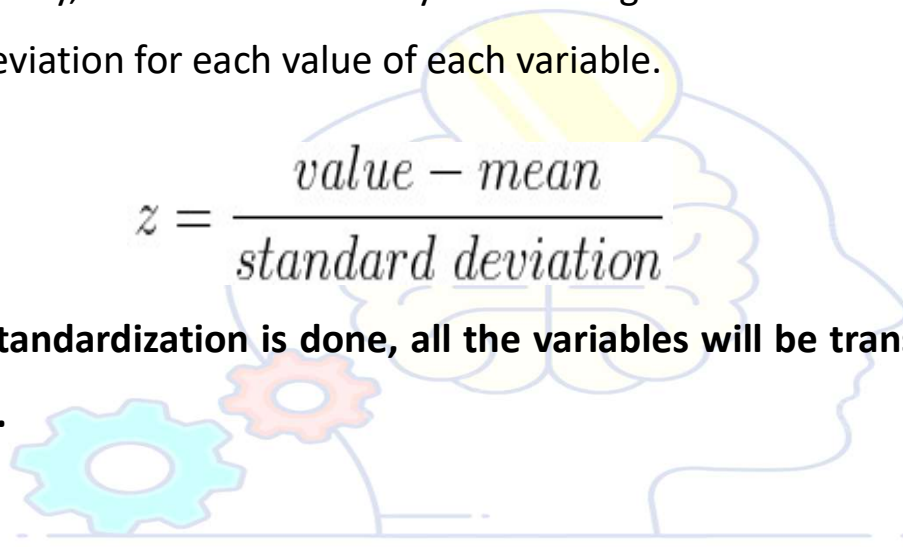
Step 1: Standardization



- Mathematically, this can be done by subtracting the mean and dividing by the standard deviation for each value of each variable.

$$z = \frac{\text{value} - \text{mean}}{\text{standard deviation}}$$

- Once the standardization is done, all the variables will be transformed to the same scale.



Machine Learning

Principal Component Analysis (PCA)

Step-by-Step Explanation

Step 2: Covariance Matrix Computation



- ❑ The aim of this step is to understand how the variables of the input data set are varying from the mean with respect to each other.
- ❑ In other words, to see if there is any relationship between them. Because sometimes, variables are highly correlated in such a way that they contain redundant information.
- ❑ The covariance matrix is a $p \times p$ symmetric matrix (where p is the number of dimensions) that has as entries the covariances associated with all possible pairs of the initial variables.

Machine Learning

Principal Component Analysis (PCA)

Step-by-Step Explanation

Step 2: Covariance Matrix Computation



- For example, for a 3-dimensional data set with 3 variables x , y , and z , the covariance matrix is a 3×3 data matrix of this form:

$$\begin{bmatrix} Cov(x, x) & Cov(x, y) & Cov(x, z) \\ Cov(y, x) & Cov(y, y) & Cov(y, z) \\ Cov(z, x) & Cov(z, y) & Cov(z, z) \end{bmatrix}$$

- Since the covariance of a variable with itself is its variance ($Cov(a, a) = Var(a)$), in the main diagonal (Top left to bottom right) we actually have the variances of each initial variable.
- And since the covariance is commutative ($Cov(a, b) = Cov(b, a)$), the entries of the covariance matrix are symmetric with respect to the main diagonal, which means that the upper and the lower triangular portions are equal.

Principal Component Analysis (PCA)

Step-by-Step Explanation

Step 2: Covariance Matrix Computation



- ❑ The aim of this step is to understand how the variables of the input data set are varying from the mean with respect to each other.
- ❑ In other words, to see if there is any relationship between them. Because sometimes, variables are highly correlated in such a way that they contain redundant information.
- ❑ The covariance matrix is a $p \times p$ symmetric matrix (where p is the number of dimensions) that has as entries the covariances associated with all possible pairs of the initial variables.

Machine Learning

Principal Component Analysis (PCA)

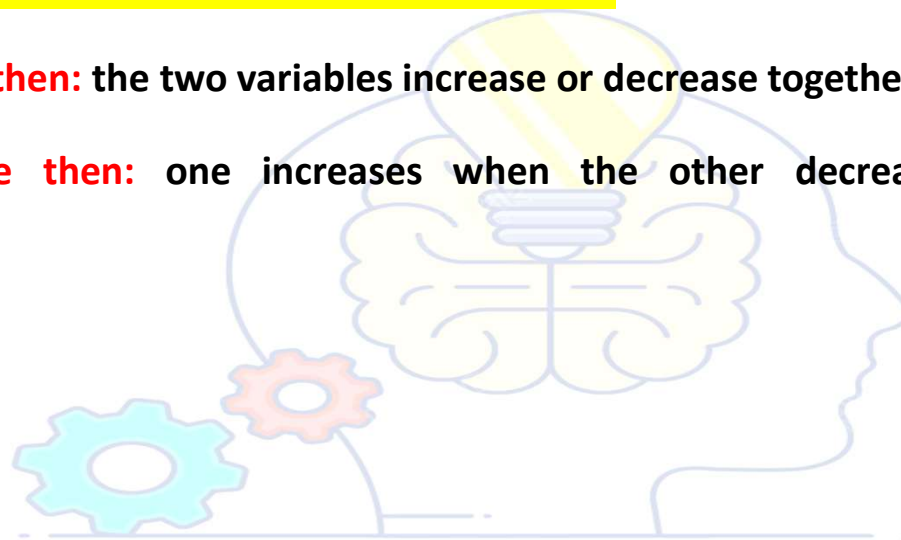
Step-by-Step Explanation

Step 2: Covariance Matrix Computation



covariances – correlations between the variables?

- ✓ **If positive then:** the two variables increase or decrease together (correlated)
- ✓ **If negative then:** one increases when the other decreases (Inversely correlated)



Machine Learning

Principal Component Analysis (PCA)

Step-by-Step Explanation



Step 3: Compute the eigenvectors and eigenvalues of the covariance matrix to identify the principal components

- ❑ **Eigenvectors** and **eigenvalues** are the linear algebra concepts that we need to compute from the covariance matrix in order to determine the principal components of the data.
- ❑ It is eigenvectors and eigenvalues who are behind all the magic of principal components because the **eigenvectors of the Covariance matrix are actually the directions of the axes where there is the most variance** (most information) and that we call **Principal Components**. And **eigenvalues are simply the coefficients attached to eigenvectors, which give the amount of variance carried in each Principal Component**.

Principal Component Analysis (PCA)

Step-by-Step Explanation



Step 3: Compute the eigenvectors and eigenvalues of the covariance matrix to identify the principal components

- ❑ **Eigenvectors** and **eigenvalues** are the linear algebra concepts that we need to compute from the covariance matrix in order to determine the principal components of the data.
- ❑ It is eigenvectors and eigenvalues who are behind all the magic of principal components because the **eigenvectors of the Covariance matrix are actually the directions of the axes where there is the most variance** (most information) and that we call **Principal Components**. And **eigenvalues are simply the coefficients attached to eigenvectors, which give the amount of variance carried in each Principal Component**.

Principal Component Analysis (PCA)

Step-by-Step Explanation



Step 3: Compute the eigenvectors and eigenvalues of the covariance matrix to identify the principal components

- By ranking your eigenvectors in order of their eigenvalues, highest to lowest, you get the principal components in order of significance.
- suppose that our data set is 2-dimensional with 2 variables x, y and that the eigenvectors and eigenvalues of the covariance matrix are as follows:

$$\begin{aligned} v_1 &= \begin{bmatrix} 0.6778736 \\ 0.7351785 \end{bmatrix} & \lambda_1 &= 1.284028 \\ v_2 &= \begin{bmatrix} -0.7351785 \\ 0.6778736 \end{bmatrix} & \lambda_2 &= 0.04908323 \end{aligned}$$

Principal Component Analysis (PCA)

Step-by-Step Explanation



Step 3: Compute the eigenvectors and eigenvalues of the covariance matrix to identify the principal components

$$v1 = \begin{bmatrix} 0.6778736 \\ 0.7351785 \end{bmatrix} \quad \lambda_1 = 1.284028$$

$$v2 = \begin{bmatrix} -0.7351785 \\ 0.6778736 \end{bmatrix} \quad \lambda_2 = 0.04908323$$

- By ranking your eigenvectors in order of their eigenvalues, highest to lowest, you get the principal components in order of significance.
- If we rank the eigenvalues in descending order, we get $\lambda_1 > \lambda_2$, which means that the eigenvector that corresponds to the first principal component (PC_1) is $v1$ and the one that corresponds to the second principal component (PC_2) is $v2$.

Principal Component Analysis (PCA)

Step-by-Step Explanation



Step 3: Compute the eigenvectors and eigenvalues of the covariance matrix to identify the principal components

$$v1 = \begin{bmatrix} 0.6778736 \\ 0.7351785 \end{bmatrix} \quad \lambda_1 = 1.284028$$

$$v2 = \begin{bmatrix} -0.7351785 \\ 0.6778736 \end{bmatrix} \quad \lambda_2 = 0.04908323$$

- ❑ After having the principal components, **to compute the percentage of variance (information) accounted for by each component, divide the eigenvalue of each component by the sum of eigenvalues.**
- ❑ For the example given, **PC₁ and PC₂ carry respectively 96 percent and 4 percent of the variance of the data.**

Machine Learning

Principal Component Analysis (PCA)

Step-by-Step Explanation

Step 4: Create a Feature Vector



- In this step, what we do is, to choose whether to keep all these components or discard those of lesser significance (of low eigenvalues), and form with the remaining ones a matrix of vectors that we call Feature vector.
- So, the *feature vector is simply a matrix* that has as **columns the eigenvectors of the components that decided to keep**. This makes it the first step towards dimensionality reduction, because if we choose to keep only p eigenvectors (components) out of n , the final data set will have only p dimensions.

Machine Learning

Principal Component Analysis (PCA)

Step-by-Step Explanation

Step 4: Create a Feature Vector



- can either form a feature vector with both of the eigenvectors v_1 and v_2 :

$$\begin{bmatrix} 0.6778736 & -0.7351785 \\ 0.7351785 & 0.6778736 \end{bmatrix}$$

- Or discard the eigenvector v_2 , which is the one of lesser significance, and form a feature vector with v_1 only:

$$\begin{bmatrix} 0.6778736 \\ 0.7351785 \end{bmatrix}$$

- Discarding the eigenvector v_2 will reduce dimensionality by 1, and will consequently cause a loss of information in the final data set. But given that v_2 was carrying only 4 percent of the information, the loss will be therefore not important and we will still have 96 percent of the information that is carried by v_1 .

Principal Component Analysis (PCA)

Step-by-Step Explanation

Step 5: Recast the Data Along the Principal Components Axes



- In this step, the aim is to use the feature vector formed using the eigenvectors of the covariance matrix, to **reorient the data from the original axes to the ones represented by the principal components** (hence the name Principal Components Analysis).
- This can be *done by multiplying the transpose of the original data set by the transpose of the feature vector.*

$$FinalDataSet = FeatureVector^T * StandardizedOriginalDataSet^T$$

Machine Learning

Principal Component Analysis (PCA)

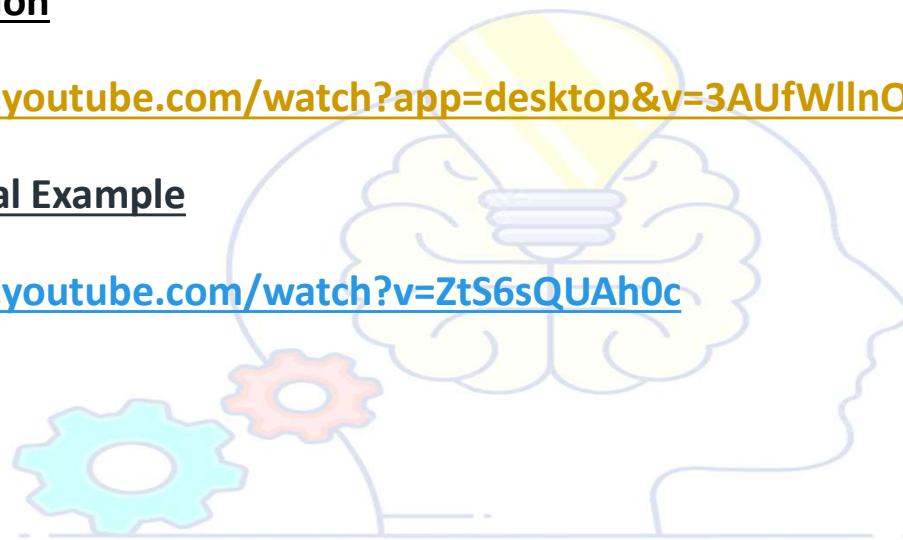


PCA Explanation

<https://www.youtube.com/watch?app=desktop&v=3AUfWllnO7c&t=155s>

PCA Numerical Example

<https://www.youtube.com/watch?v=ZtS6sQUAh0c>



Machine Learning

Ensemble Learning



Machine Learning

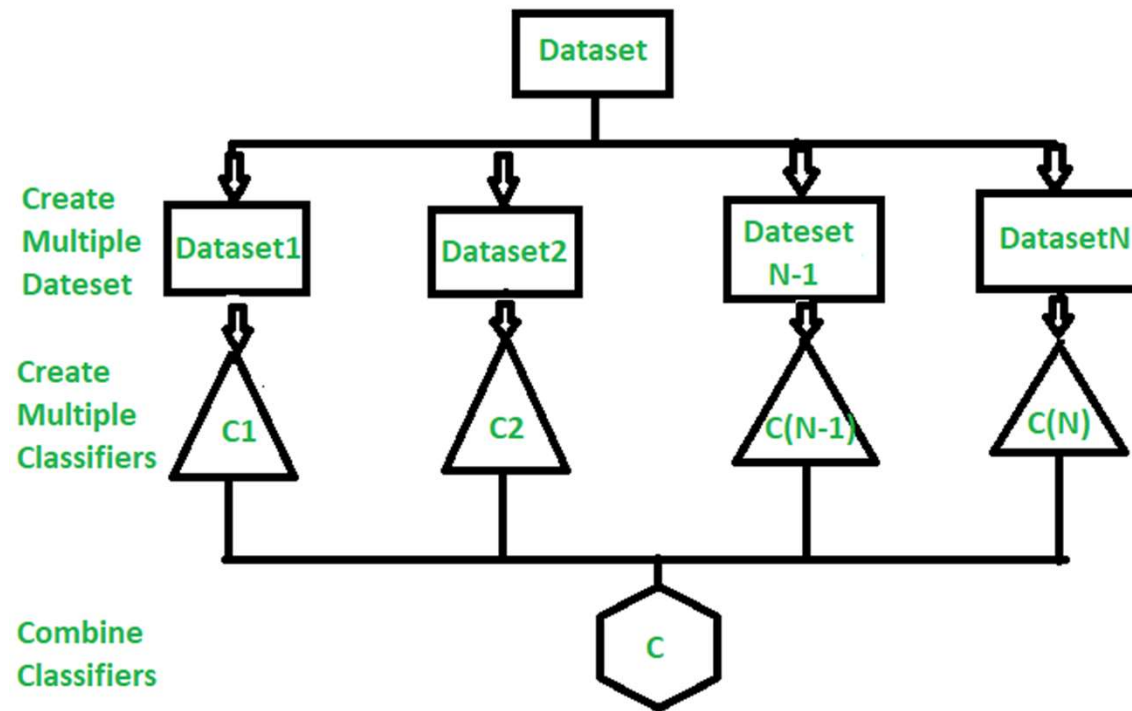
Ensemble Learning



- Ensemble learning helps improve machine learning results by combining several models.
- This approach allows the production of better predictive performance compared to a single model.
- Basic idea is to learn a set of classifiers (experts) and to allow them to vote.
- **Advantage** : Improvement in predictive accuracy.
- **Disadvantage** : It is difficult to understand an ensemble of classifiers.

Machine Learning

Ensemble Learning



Ensemble Learning –Why?



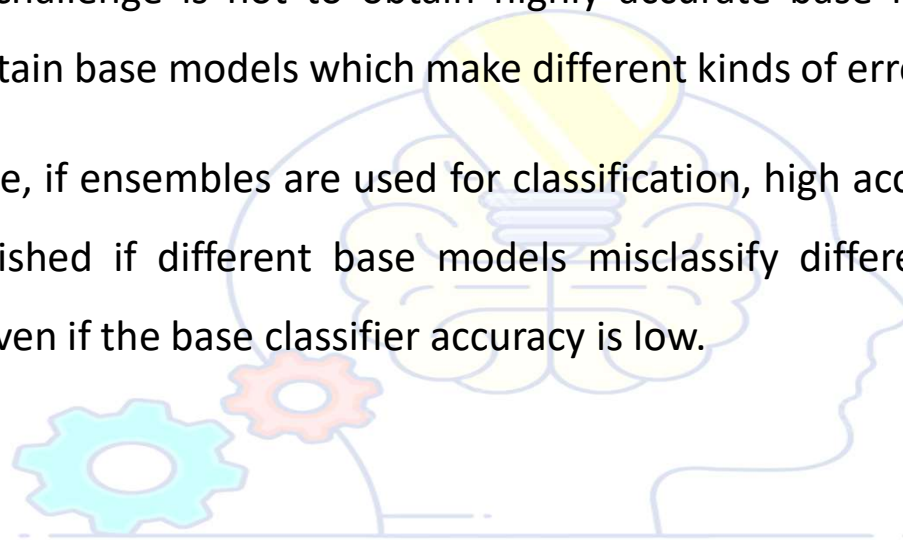
- **Statistical Problem** –The Statistical Problem arises when the hypothesis space is too large for the amount of available data. Hence, there are many hypotheses with the same accuracy on the data and the learning algorithm chooses only one of them! There is a risk that the accuracy of the chosen hypothesis is low on unseen data!
- **Computational Problem** –The Computational Problem arises when the learning algorithm cannot guarantee finding the best hypothesis.
- **Representational Problem** – The Representational Problem arises when the hypothesis space does not contain any good approximation of the target class(es).

Ensemble Learning

Main Challenge for Developing Ensemble Models?



- The main challenge is not to obtain highly accurate base models, but rather to obtain base models which make different kinds of errors.
- For example, if ensembles are used for classification, high accuracies can be accomplished if different base models misclassify different training examples, even if the base classifier accuracy is low.



Machine Learning

Ensemble Learning

Main Challenge for Developing Ensemble Models?

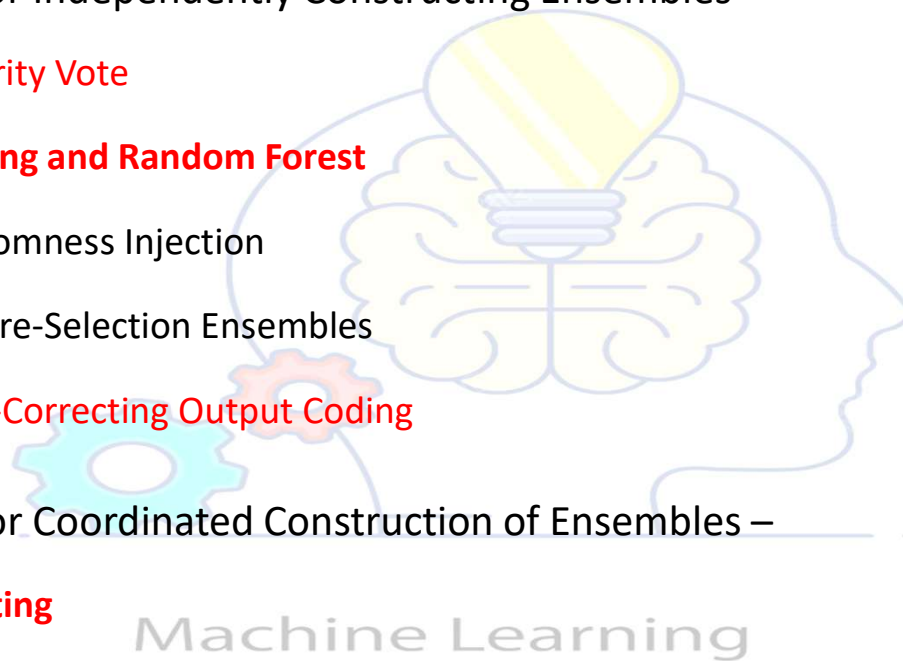


➤ Methods for Independently Constructing Ensembles –

- ☐ Majority Vote
- ☐ Bagging and Random Forest
- ☐ Randomness Injection
- ☐ Feature-Selection Ensembles
- ☐ Error-Correcting Output Coding

➤ Methods for Coordinated Construction of Ensembles –

- ☐ Boosting
- ☐ Stacking



Ensemble Learning –

Voting Classifier



- A voting classifier is a machine learning model that gains experience by training on a collection of several models and **forecasts an output (class) based on the class with the highest likelihood of becoming the output.**
- To forecast the output class based on the largest majority of votes, it averages the results of each classifier provided into the voting classifier.
- The concept is to build a single model that learns from various models and predicts output based on their aggregate majority of votes for each output class, rather than building separate specialized models and determining the accuracy for each of them.

Ensemble Learning – Voting Classifier



There are primarily **two different types of voting classifiers**:

- **Hard Voting**: In hard voting, the predicted output class is a class with the highest majority of votes, i.e., the class with the highest probability of being predicted by each classifier.
- For example, let's say classifiers predicted the output classes as (Cat, Dog, Dog). As the classifiers predicted class "dog" a maximum number of times, we will proceed with Dog as our final prediction.

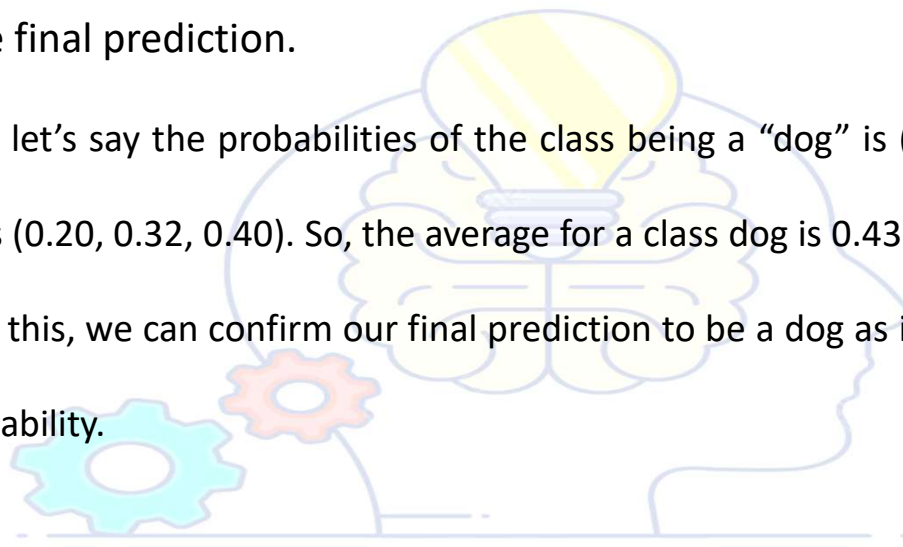
Machine Learning

Ensemble Learning – Voting Classifier



➤ **Soft Voting:** In this, the average probabilities of the classes determine which one will be the final prediction.

➤ For example, let's say the probabilities of the class being a "dog" is (0.30, 0.47, 0.53) and a "cat" is (0.20, 0.32, 0.40). So, the average for a class dog is 0.4333, and the cat is 0.3067, from this, we can confirm our final prediction to be a dog as it has the highest average probability.



Machine Learning

Ensemble Learning – Error Correcting Output Codes(ECOC)



- Error Correcting Output Codes (ECOC) is a powerful technique used in machine learning to enhance the accuracy of classification algorithms, especially in multi-class classification problems.
- ECOC is a strategy that decomposes a multi-class classification problem into multiple binary classification problems.
- By assigning binary codes to each class, ECOC leverages error correction principles to handle classification errors more effectively.

Machine Learning

Ensemble Learning –

Error Correcting Output Codes(ECOC)



- While binary classification is relatively straightforward, multi-class classification can be more complex.
- ECOC is a technique designed to simplify and improve multi-class classification by decomposing it into multiple binary classification problems.
- ECOC was first introduced by Dietterich and Bakiri in 1995 as a method to enhance the performance of classifiers by leveraging error-correcting codes.
- The core idea is to represent each class with a unique binary code and then train binary classifiers to distinguish between these codes.
- **The concept of ECOC stems from the domain of error-correcting codes in information theory, where redundancy is added to data to detect and correct errors during transmission.**

Ensemble Learning –

Error Correcting Output Codes(ECOC)



Theoretical Foundations of ECOC

- The theoretical foundation of ECOC lies in error-correcting codes, which are used in digital communication to detect and correct errors. In the context of machine learning, ECOC uses these codes to create a robust classification system.
- **Error-Correcting Codes:** Error-correcting codes are mathematical constructs that add redundancy to data to detect and correct errors. Common examples include Hamming codes and Reed-Solomon codes. These codes are characterized by their ability to correct a certain number of errors based on the code length and redundancy.

Ensemble Learning –

Error Correcting Output Codes(ECOC)



Theoretical Foundations of ECOC

- **ECOC in Machine Learning:** In ECOC, each class in a multi-class problem is assigned a unique binary code, known as a codeword. These codewords are arranged in a code matrix, where each row represents a class, and each column represents a binary classifier. The binary classifiers are trained to distinguish between the columns of the code matrix.

Machine Learning

Ensemble Learning –

Error Correcting Output Codes(ECOC)

How Error Correcting Output Codes(ECOC) Works?



The process of ECOC involves two main steps: encoding and decoding.

❏ Encoding

- **Code Matrix Construction:** The first step is to construct a code matrix, where each row represents a class, and each column represents a binary classifier. The entries in the matrix are typically -1, 0, or 1, where -1 and 1 indicate the two classes for the binary classifier, and 0 indicates that the classifier does not consider that class.
- **Binary Classifier Training:** Each column of the code matrix corresponds to a binary classification problem. Binary classifiers are trained using the data, where the labels are determined by the entries in the code matrix.

Ensemble Learning –

Error Correcting Output Codes(ECOC)



How Error Correcting Output Codes(ECOC) Works?

The process of ECOC involves two main steps: encoding and decoding.

❑ Decoding

- **Prediction:** For a new instance, each binary classifier makes a prediction, resulting in a binary vector.
- **Codeword Matching:** The predicted binary vector is compared to the codewords in the code matrix. The class with the closest codeword (usually measured by Hamming distance) is selected as the predicted class.

Machine Learning

Ensemble Learning –

Error Correcting Output Codes(ECOC)



Types of ECOC

- ❑ **One-vs-All (OvA)**: In the One-vs-All approach, each class is compared against all other classes. This results in a code matrix where each column has one class labeled as 1 and all others as -1. This method is simple but may not be optimal for all problems.
- ❑ **One-vs-One (OvO)**: In the One-vs-One approach, each pair of classes is compared, resulting in a code matrix where each column represents a binary classifier for a pair of classes. This method can be more accurate but requires training more classifiers.
- ❑ **Dense and Sparse Codes**: Dense codes use a larger number of binary classifiers, resulting in more robust error correction but higher computational cost. Sparse codes use fewer classifiers, reducing computational cost but potentially sacrificing some robustness.

Ensemble Learning –

Error Correcting Output Codes(ECOC)



Implementation of ECOC using Python

➤ Step 1: Import Libraries

```
import numpy as np
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.multiclass import OutputCodeClassifier
from sklearn.metrics import accuracy_score
```

Machine Learning

Ensemble Learning –

Error Correcting Output Codes(ECOC)



Implementation of ECOC using Python

➤ Step 2: Load and Prepare Data

```
# Load the Iris dataset
```

```
data = load_iris()
```

```
X = data.data
```

```
y = data.target
```

```
# Split the data into training and testing sets
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
```

Machine Learning

Ensemble Learning –

Error Correcting Output Codes(ECOC)



Implementation of ECOC using Python

➤ Step 3: Train ECOC Classifier

```
# Create an SVM classifier
svm = SVC(kernel='linear')

# Create an ECOC classifier
ecoc = OutputCodeClassifier(estimator=svm, code_size=2, random_state=42)

# Train the ECOC classifier
ecoc.fit(X_train, y_train)
```

Machine Learning

Ensemble Learning –

Error Correcting Output Codes(ECOC)



Implementation of ECOC using Python

➤ Step 4: Evaluate the Classifier

```
# Make predictions
y_pred = ecoc.predict(X_test)
# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)
print(f'Accuracy: {accuracy * 100:.2f}%')
```

This simple implementation demonstrates how to use ECOC with an SVM classifier to solve a multi-class classification problem.

Ensemble Learning –

Error Correcting Output Codes(ECOC)

Advantages of Error Correcting Output Codes(ECOC)



- **Robustness to Errors:** ECOC enhances robustness by allowing for error correction. If a few binary classifiers make incorrect predictions, the correct class can still be identified due to the redundancy in the code matrix.
- **Improved Generalization:** By breaking down the multi-class problem into simpler binary problems, ECOC can improve the generalization capabilities of classifiers, particularly when dealing with imbalanced datasets or noisy data.
- **Flexibility:** ECOC can be used with various types of base classifiers, such as decision trees, support vector machines, or neural networks. This flexibility allows practitioners to choose the most suitable classifiers for their specific problem.

Ensemble Learning –

Error Correcting Output Codes(ECOC)

Applications of Error Correcting Output Codes(ECOC)



➤ Image Classification

- ECOC has been widely used in image classification tasks where distinguishing between multiple classes can be challenging due to similarities between classes and variations within classes.

❖ Example: Handwritten Digit Recognition

➤ Text Classification

- In natural language processing, ECOC can improve text classification by breaking down the complex task of categorizing text into manageable binary classifications.

❖ Example: News Article Classification

Ensemble Learning –

Error Correcting Output Codes(ECOC)

Applications of Error Correcting Output Codes(ECOC)



➤ **Bioinformatics**

- ECOC is applied in bioinformatics for tasks such as gene expression classification, where distinguishing between various biological states or conditions is crucial.

- ❖ **Example: Cancer Type Classification**

➤ **Speech Recognition**

- ECOC can be beneficial in speech recognition tasks where distinguishing between multiple phonemes or words is necessary.

- ❖ **Example: Phoneme Classification**

Machine Learning

Ensemble Learning –

Error Correcting Output Codes(ECOC)

Challenges and Considerations



- **Code Matrix Design:** Designing an effective code matrix is critical for the success of ECOC. The code matrix should balance between having enough redundancy for error correction and not being overly complex.
- **Computational Complexity:** Training multiple binary classifiers can be computationally intensive, particularly for large datasets. Efficient algorithms and parallel computing can mitigate this challenge.
- **Base Classifier Performance:** The overall performance of ECOC depends on the accuracy of the individual binary classifiers. Poor performance of base classifiers can lead to suboptimal results, even with error correction.

Ensemble Learning



- In machine learning, for building solid and reliable models, prediction accuracy is the key factor.
- Ensemble learning is a supervised machine-learning technique that combines multiple models to build a more powerful and robust model.
- The idea is that by combining the strengths of multiple models, we can create a model that is **more robust and less likely to overfit the data.**
- It can be used for both classification and regression tasks.

Machine Learning

Ensemble Learning



➤ Ensemble learning techniques can be categorized in three ways:

❑ **Bagging (Bootstrap Aggregating)**

❑ **Boosting**

❑ **Stacking (Stacked Generalization)**

➤ Bagging is a supervised machine-learning technique, and it can be used for both regression and classification tasks.

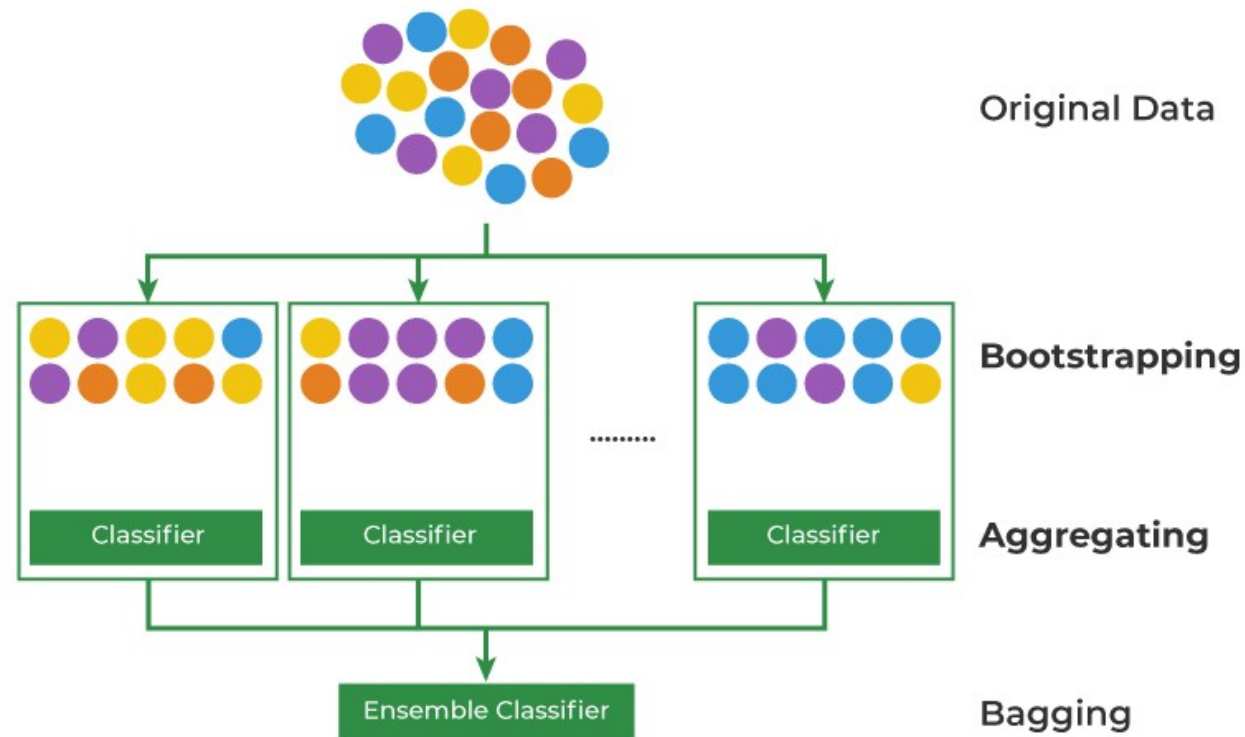
Machine Learning

Ensemble Learning – Bagging Classifier



- Bagging (or Bootstrap aggregating) is a type of ensemble learning in which multiple base models are trained independently and in parallel on different subsets of the training data.
- Each subset is generated using bootstrap sampling, in which data points are picked at random with replacement.
- In the case of the bagging classifier, the final prediction is made by aggregating the predictions of the all-base model using majority voting.
- In the models of regression, the final prediction is made by averaging the predictions of the all-base model, and that is known as bagging regression.

Ensemble Learning – Bagging Classifier



Bagging helps improve accuracy and reduce overfitting, especially in models that have high variance.

Ensemble Learning – Bagging Classifier



How does Bagging Classifier Work?

The basic steps of how a bagging classifier works are as follows:

1. **Bootstrap Sampling:** In Bootstrap Sampling randomly 'n' subsets of original training data are sampled with replacement. This step ensures that the base models are trained on diverse subsets of the data, as some samples may appear multiple times in the new subset, while others may be omitted. It reduces the risks of overfitting and improves the accuracy of the model.

Machine Learning

Ensemble Learning – Bagging Classifier



How does Bagging Classifier Work?

- 2. Base Model Training:** In bagging, multiple base models are used. After the Bootstrap Sampling, each base model is independently trained using a specific learning algorithm, such as decision trees, support vector machines, or neural networks on a different bootstrapped subset of data. These models are typically called “Weak learners” because they may not be highly accurate on their own. Since the base model is trained independently of different subsets of data. To make the model computationally efficient and less time-consuming, the base models can be trained in parallel.

Ensemble Learning – Bagging Classifier



How does Bagging Classifier Work?

3. **Aggregation:** Once all the base models are trained, it is used to make predictions on the unseen data i.e. the subset of data on which that base model is not trained. In the bagging classifier, the predicted class label for the given instance is chosen based on the majority voting. The class which has the majority voting is the prediction of the model.

Machine Learning

Ensemble Learning – Bagging Classifier



How does Bagging Classifier Work?

4. **Out-of-Bag (OOB) Evaluation:** Some samples are excluded from the training subset of particular base models during the bootstrapping method. These “out-of-bag” samples can be used to estimate the model’s performance without the need for cross-validation.
5. **Final Prediction:** After aggregating the predictions from all the base models, Bagging produces a final prediction for each instance.

Machine Learning



Ensemble Learning – Bagging Classifier

Advantages of Bagging Classifier

- **Improved Predictive Performance:** Bagging Classifier often outperforms single classifiers by reducing overfitting and increasing predictive accuracy. By combining multiple base models, it can better generalize to unseen data.
- **Robustness:** Bagging reduces the impact of outliers and noise in the data by aggregating predictions from multiple models. This enhances the overall stability and robustness of the model.
- **Reduced Variance:** Since each base model is trained on different subsets of the data, the aggregated model's variance is significantly reduced compared to an individual model.

Ensemble Learning – Bagging Classifier

Advantages of Bagging Classifier

- **Parallelization:** Bagging allows for parallel processing, as each base model can be trained independently. This makes it computationally efficient, especially for large datasets.
- **Flexibility:** Bagging Classifier is a versatile technique that can be applied to a wide range of machine learning algorithms, including decision trees, random forests, and support vector machines.



Machine Learning



Ensemble Learning – Bagging Classifier

Disadvantages of Bagging

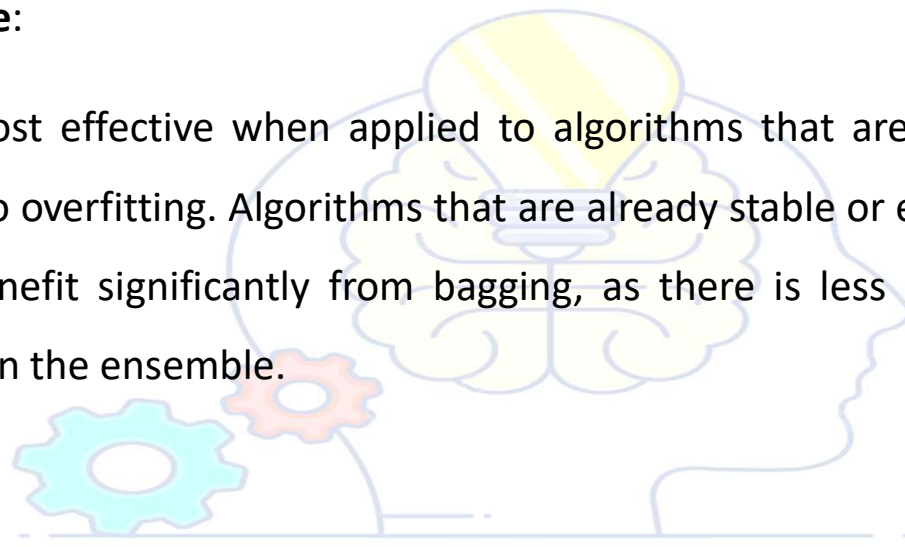
- **Loss of Interpretability:** Bagging involves aggregating predictions from multiple models, making it harder to interpret the individual contributions of each base model. This can limit the ability to derive precise business insights from the ensemble.
- **Computationally Expensive:** As the number of iterations (bootstrap samples) increases, the computational cost of bagging also grows. This makes it less suitable for real-time applications where efficiency and speed are crucial. Efficient parallel processing or distributed systems are often required for handling large datasets and numerous iterations.

Ensemble Learning – Bagging Classifier

Disadvantages of Bagging

➤ Less Flexible:

Bagging is most effective when applied to algorithms that are less stable or more prone to overfitting. Algorithms that are already stable or exhibit low bias might not benefit significantly from bagging, as there is less variance to be reduced within the ensemble.



Machine Learning



Ensemble Learning – Bagging Classifier

Applications of Bagging Classifier

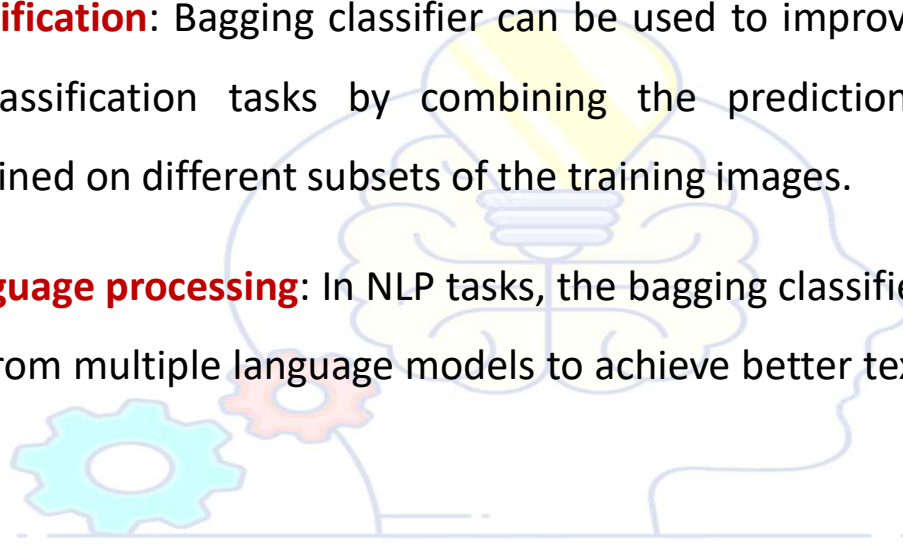
- **Fraud Detection:** Bagging Classifier can be used to detect fraudulent transactions by aggregating predictions from multiple fraud detection models.
- **Spam filtering:** Bagging classifier can be used to filter spam emails by aggregating predictions from multiple spam filters trained on different subsets of the spam emails.
- **Credit scoring:** Bagging classifier can be used to improve the accuracy of credit scoring models by combining the predictions of multiple models trained on different subsets of the credit data.

Ensemble Learning – Bagging Classifier

Applications of Bagging Classifier



- **Image Classification:** Bagging classifier can be used to improve the accuracy of image classification tasks by combining the predictions of multiple classifiers trained on different subsets of the training images.
- **Natural language processing:** In NLP tasks, the bagging classifier can combine predictions from multiple language models to achieve better text classification results.



Machine Learning

Ensemble Learning –

Bagging : Random Forest Trees



- A **random forest is an ensemble learning method** that combines the predictions from multiple decision trees to produce a more accurate and stable prediction.
- It is a **type of supervised learning algorithm** that can be used for both classification and regression tasks.
- Every decision tree has high variance, but when we combine all of them in parallel then the resultant variance is low as each decision tree gets perfectly trained on that particular sample data, and hence the output doesn't depend on one decision tree but on multiple decision trees.

Ensemble Learning –

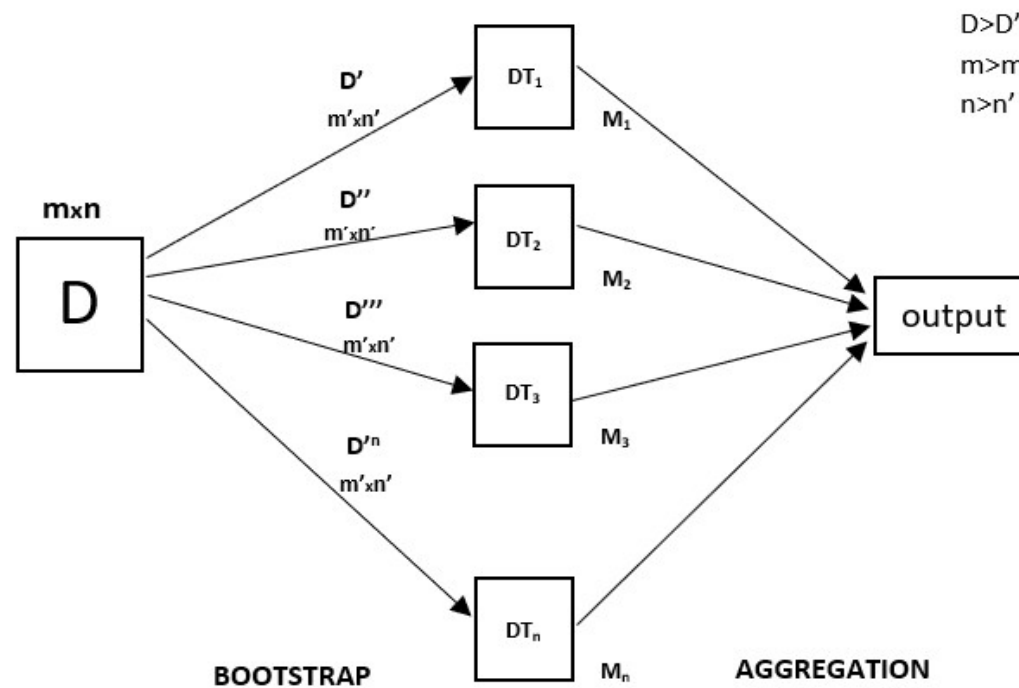
Bagging : Random Forest Trees



- In the case of a classification problem, the final output is taken by using the majority voting classifier. In the case of a regression problem, the final output is the mean of all the outputs. This part is called Aggregation.
- A technique called Bootstrap and Aggregation, commonly known as bagging.
- The basic idea behind this is to combine multiple decision trees in determining the final output rather than relying on individual decision trees.
- Random Forest has multiple decision trees as base learning models. We randomly perform row sampling and feature sampling from the dataset forming sample datasets for every model. This part is called **Bootstrap**.

Ensemble Learning –

Bagging : Random Forest Trees



Ensemble Learning – Boosting



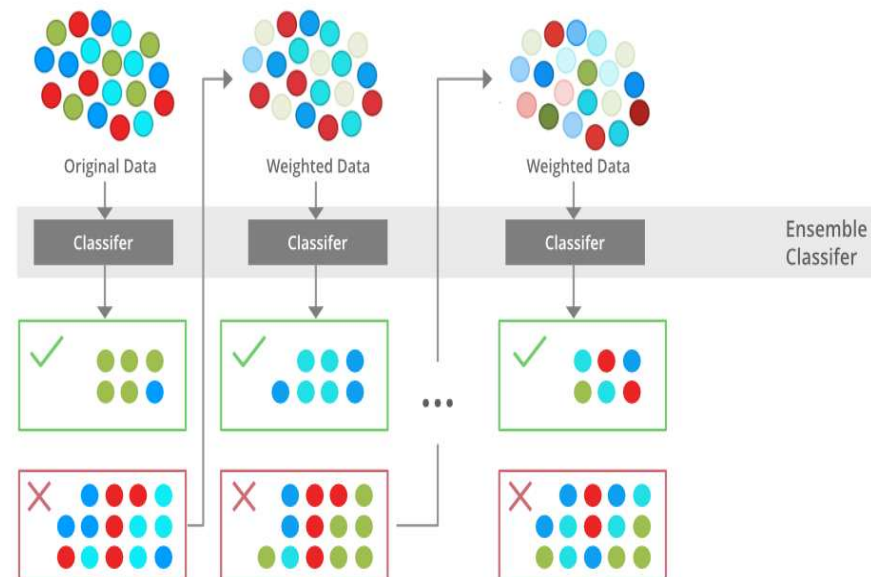
- A single weak model may not be enough for our complex problems such cases we aggregate various weak models to make a powerful and more accurate model for our problem this process of aggregating several small problems to create a strong model is what we do in boosting.
- Boosting is an ensemble modeling technique that attempts to build a strong classifier from the number of weak classifiers.
- It is done by building a model by using weak models in series.
- Firstly, a model is built from the training data.

Ensemble Learning – Boosting



➤ Then the second model is built which tries to correct the errors present in the first model.

➤ This procedure is continued and models are added until either the complete training data set is predicted correctly or the maximum number of models are added.



Ensemble Learning – Boosting



Advantages of Boosting

- **Improved Accuracy** – Boosting can improve the accuracy of the model by combining several weak models' accuracies and averaging them for regression or voting over them for classification to increase the accuracy of the final model.
- **Robustness to Overfitting** – Boosting can reduce the risk of overfitting by reweighting the inputs that are classified wrongly.
- **Better handling of imbalanced data** – Boosting can handle the imbalance data by focusing more on the data points that are misclassified.
- **Better Interpretability** – Boosting can increase the interpretability of the model by breaking the model decision process into multiple processes.

Ensemble Learning – Boosting

Types Of Boosting Algorithms



1. Gradient Boosting:

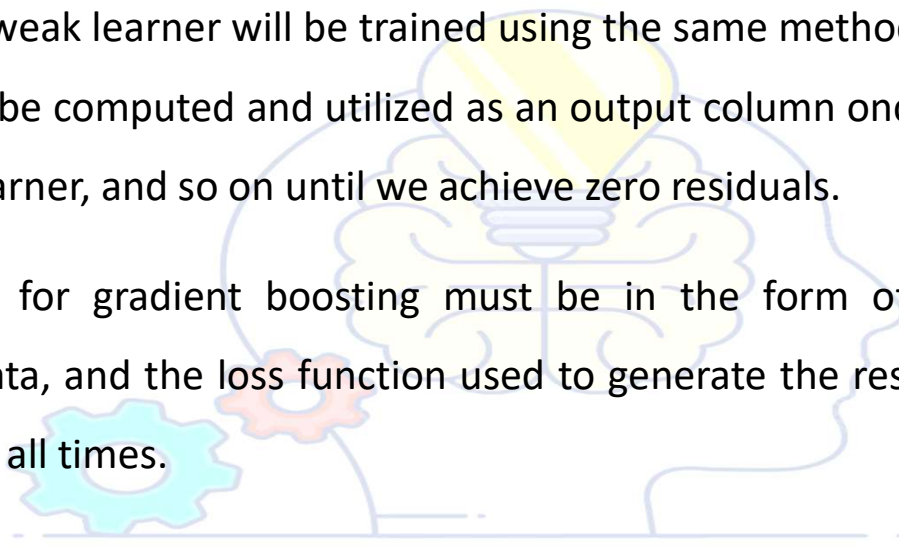
- It is a boosting technique that builds a final model from the sum of several weak learning algorithms that were trained on the same dataset. It operates on the idea of stagewise addition.
- The first weak learner in the gradient boosting algorithm will not be trained on the dataset; instead, it will simply return the mean of the relevant column.
- The residual for the first weak learner algorithm's output will then be calculated and used as the output column or target column for the next weak learning algorithm that will be trained.

Ensemble Learning – Boosting

Types Of Boosting Algorithms

1. Gradient Boosting:

- The second weak learner will be trained using the same methodology, and the residuals will be computed and utilized as an output column once more for the third weak learner, and so on until we achieve zero residuals.
- The dataset for gradient boosting must be in the form of numerical or categorical data, and the loss function used to generate the residuals must be differential at all times.



Machine Learning

Ensemble Learning – Boosting

Types Of Boosting Algorithms



2. XGBoost:

- XGBoost is another boosting machine learning approach.
- The full name of the XGBoost algorithm is the **eXtreme Gradient Boosting algorithm**, which is an extreme variation of the previous gradient boosting technique.
- The key distinction between XGBoost and GradientBoosting is that XGBoost applies a regularization approach.

Machine Learning

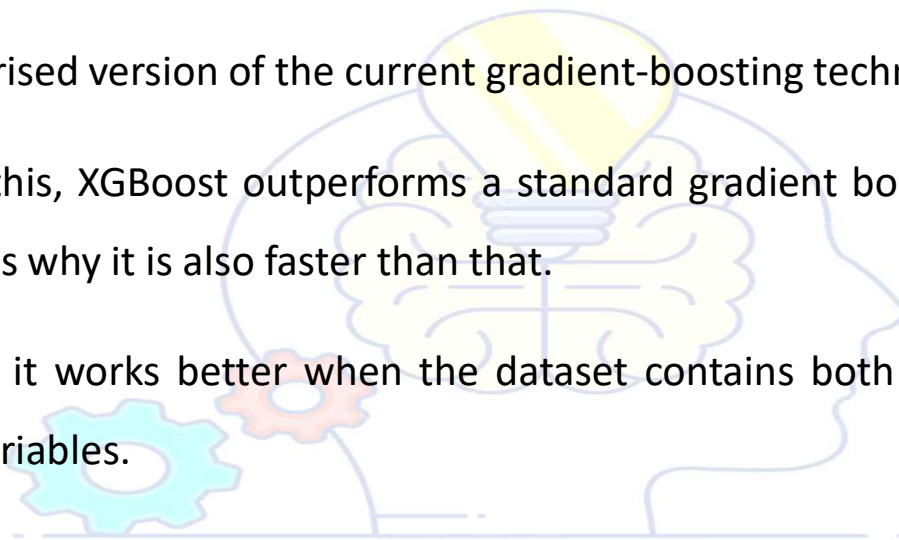
Ensemble Learning – Boosting

Types Of Boosting Algorithms



2. XGBoost:

- It is a regularised version of the current gradient-boosting technique.
- Because of this, XGBoost outperforms a standard gradient boosting method, which explains why it is also faster than that.
- Additionally, it works better when the dataset contains both numerical and categorical variables.



Machine Learning

Ensemble Learning – Boosting

Types Of Boosting Algorithms



3. Adaboost:

- AdaBoost is a boosting algorithm that also works on the principle of the stagewise addition method where multiple weak learners are used for getting strong learners.
- The value of the alpha parameter, in this case, will be indirectly proportional to the error of the weak learner, Unlike Gradient Boosting in XGBoost, the alpha parameter calculated is related to the errors of the weak learner, here the value of the alpha parameter will be indirectly proportional to the error of the weak learner.

Ensemble Learning – Boosting

Types Of Boosting Algorithms



4. CatBoost:

- The growth of decision trees inside CatBoost is the primary distinction that sets it apart from and improves upon competitors.
- The decision trees that are created in CatBoost are symmetric.
- As there is a unique sort of approach for handling categorical datasets, CatBoost works very well on categorical datasets compared to any other algorithm in the field of machine learning.

Machine Learning

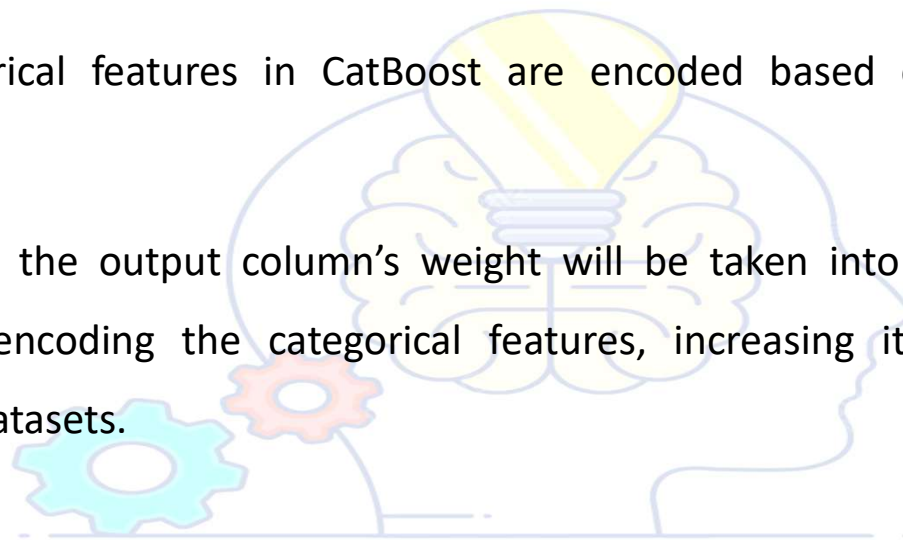
Ensemble Learning – Boosting

Types Of Boosting Algorithms



4. CatBoost:

- The categorical features in CatBoost are encoded based on the output columns.
- As a result, the output column's weight will be taken into account while training or encoding the categorical features, increasing its accuracy on categorical datasets.



Machine Learning

Ensemble Learning – Stacking



➤ Stacking is a **way of ensembling classification or regression models** it consists of **two-layer estimators**.

- The **first layer** consists of all the baseline models that are used to predict the outputs on the test datasets.
- The **second layer** consists of Meta-Classifier or Regressor which takes all the predictions of baseline models as an input and generate new predictions.
- Can improve the existing accuracy that is shown by individual models.
- Can get most of the Stacked models by choosing diverse algorithms in the first layer of architecture as different algorithms capture different trends in training data by combining both of the models can give better and accurate results.

Ensemble Learning – Stacking

Architecture

